**UNIVERSITETI POLITEKNIK – TIRANË**
**Fakulteti i Teknologjisë së Informacionit**
**Sheshi Nënë Tereza, 1 – Tiranë**
**Tel/Fax : +355 4 2278 159**

# PROJEKT – DIPLOMË
# Cikli i Parë i Studimeve
# Bachelor
# në
# Inxhinieri Elektronike

***TEMA:*** Dizajni dhe Implementimi i një Arkitekture Procesori RISC-V në FPGA

| | |
|---|---|
| ***DEKANI*** | ***Prof. Dr. Elinda Meçe*** |
| ***PËRGJEGJËSI I DEPARTAMENTIT*** | ***Prof. Dr. Aleksandër Biberaj*** |
| ***UDHËHEQËSI*** | ***Prof.Msc. Gani Demiraj*** |
| ***DIPLOMANTI*** | ***Samuel Koçi*** |

***Tiranë 2025-2026***

**REPUBLIKA E SHQIPËRISË**
**UNIVERSITETI POLITEKNIK I TIRANËS**
**FAKULTETI I TEKNOLOGJISË SË INFORMACIONIT**
**DEPARTAMENTI I ELEKTRONIKËS DHE TELEKOMUNIKACIONIT**
Sheshi "Nënë Tereza", Nr. 1, Tiranë
Tel. dhe Fax : (+355) 4 2278 159

**DEKANI**

***Prof. Dr. Elinda Meçe***

# FLETË – DETYRË

## mbi PROJEKT – DIPLOMËN
## Cikli i Parë i Studimeve
## Bachelor
## në
## Inxhinieri Elektronike

Studenti : **Samuel Ardian Koçi** Nr. i Regj. PO523A120470

Departamenti i Elektronikës dhe Telekomunikacionit

### I. Tema e Projekt – Diplomës

Dizajni dhe Implementimi i një Arkitekture Procesori RISC-V në FPGA

### II. Afati i dorëzimit të Projekt- Diplomës

**24.06.2026**

---

### III. Të dhëna mbi Projekt- Diplomën

1. Platforma Harduerike (FPGA): Përdorimi i kornizës së zhvillimit AMD/Xilinx
2. Seti i Instruksioneve (ISA)
3. Mjedisi i Zhvillimit RTL
4. Mjedisi Softuerik Bare-Metal: RISC-V GCC
5. Korniza e Validimit (Benchmark): EEMBC Coremark

### IV. Përmbajtja e Projekt- Diplomës

- **Relacioni**

1. Analiza Teorike dhe Dizajni RTL Single-Cycle
2. Implementimi i Memories dhe Ndërfaqes MMIO
3. Projektimi i Komunikimit Periferik dhe Kohës
4. Portimi i Softuerit dhe Metodologjia e Debug-imit

5. Analiza e Rezultateve dhe Performancës

**V. Kontrolloi në Departament (studenti është i detyruar që me materialet e përgatitura në atë kohë të paraqitet në Departament)**

**1.** 17.04.2026 Kontrolloi *MSc. Gani Demiraj*

**2.** 20.05.2026 Kontrolloi *MSc. Gani Demiraj*

**3.** 24.06.2026 Kontrolloi *Prof. Dr. Aleksandër Biberaj*

Udhëheqësi MSc. Gani Demiraj
(titulli, emri, mbiemri)

Data e miratimit të temës 09.03.2026

**Përgjegjësi i Departamentit**

**Prof. Dr. Aleksandër Biberaj**

# PËRMBAJTJA

LISTA E FIGURAVE....7
LISTA E TABELAVE....7
LISTE E DIAGRAMEVE....8
ABSTRAKTI....9
METODOLOGJIA E PËRDORUR....10
Kapitulli 1: Hyrje....11
1.1 Konteksti dhe Motivimi....11
1.2 Qëllimi i Punimit....11
1.3 Problematika....12
1.4 Objektivat e Punimit....12
1.5 Struktura e Punimit....13
1.6 Kontekst dhe Krahasim me Projekte të Ngjashme Open-Source....14
Kapitulli 2: Bazat Teorike dhe Arkitektura RISC-V....15
2.1 Çfarë është RISC-V?....15
2.2 Arkitektura RV32I dhe Zgjerimi M....15
2.3 Arkitektura Load-Store dhe Ndërveprimi me Memorien....17
2.4 Regjistrat dhe Konventa e Thirrjes (ABI)....17
2.5 Kodimi i Sinjalit alu_control....19
2.6 Shembull i Plotë: Dekodimi Hap-pas-Hapi i një Instruksioni....20
Kapitulli 3: Arkitektura dhe Dizajni i Sistemit Hardware....21
3.1 Përmbledhje e Arkitekturës së Sistemit....21
3.2 Rruga e të Dhënave (Datapath) dhe Dizajni Single-Cycle....22
3.2.1 Karakteristikat e Kohës (Timing) të Register File....22
3.3 Njësia Aritmetike-Logjike (ALU)....24
3.4 Njësia e Kontrollit (Control Unit)....24

3.5 Sinkronizimi me Memorien BRAM dhe Makina e Gjendjeve (FSM) ..... 25
3.6 Ndërfaqja e Memories dhe Instruksioni LBU ..... 27
3.7 Sistemi Memory-Mapped I/O (MMIO) dhe Periferikët ..... 28
3.8 Pse Hazardet Klasike të Pipeline-it Nuk Aplikohen (dhe Çfarë Ndryshon nëse Shtohet Pipeline) ..... 28
3.9 Harvard kundër Von Neumann: Implikimet Praktike të Zgjedhjes Arkitekturore ..... 29
3.10 Diagramet Timing: Ekzekutimi Cikël-pas-Cikli ..... 30
3.10.1 R-type / BRANCH (2 Cikle) ..... 30
3.10.2 LOAD nga RAM (4 Cikle) ..... 31
3.10.3 STORE (2 Cikle, "Fire-and-Forget") ..... 31
Kapitulli 4: Mjedisi Softuerik dhe Portimi i CoreMark ..... 33
4.1 Mjedisi i Zhvillimit (Toolchain) dhe Strategjia e Kompilimit ..... 33
4.2 Ekzekutimi "Bare-Metal" dhe Skedari Linker ..... 34
4.3 Sekuenca e Zgjimit (Boot Sequence) dhe Funksioni _start ..... 35
4.4 Portimi i Kornizës EEMBC CoreMark ..... 36
4.6 Shërbimi i Printimit të Formatuar (ee_printf) ..... 37
4.6.1 Llogaritja e Parametrit N të Baud Rate Generator-it ..... 37
4.7 Harta e Memories (Memory Map) ..... 38
Kapitulli 5: Metodologjia e Testimit dhe Rezultatet ..... 39
5.1 Mjedisi i Testimit në FPGA ..... 39
5.2 Testimi i Instruksioneve dhe Komponentëve ..... 39
5.3 Ekzekutimi i CoreMark ..... 40
5.4 Analiza e Performancës dhe Rezultatet Finale ..... 41
5.5 Shpjegimi Teknik i Algoritmeve të Benchmark-ut ..... 43
5.6 Përdorimi i Burimeve FPGA (Resource Utilization) ..... 44
Kapitulli 6: Arkitektura e Kodit dhe Struktura e Projektit ..... 46
6.1 Hierarkia e Skedarëve (File Structure) ..... 46

6.2 Përshkrimi i Modulave SystemVerilog ..... 46
6.3 Karakteristikat Sintetizuese të Secilit Modul ..... 48
6.3.1 Shembull i Plotë: Ekzekutimi i një Instruksioni STORE ..... 48
6.4 Skriptet e Simulimit dhe Verifikimit ..... 49
6.5 Struktura e një Testbench-i Tipik ..... 50
Kapitulli 7: Sfidat Teknike dhe Zgjidhjet e Implementimit ..... 52
7.8 Metrikat e Procesit të Debug-imit ..... 53
7.1 Problemi i X-Propagation nga Memoria .bss e Pa-inicializuar ..... 53
7.2 Bug-u "Off-by-One Cycle" në FSM (X-Propagation Sekondare) ..... 55
7.3 Çështjet e Toolchain-it: Global Pointer, _start, dhe Linker Script ..... 56
7.4 Krashimi i Vivado gjatë Opt Design (Power Optimization) ..... 57
7.5 Loop Infinit në core_bench_state (Bug në Versionin e Thjeshtuar) ..... 58
7.6 Nga Algoritmi i Thjeshtuar tek CoreMark Zyrtar EEMBC ..... 59
7.7 Verifikimi Empirik i Hipotezave: Rasti i Loop-it Infinit ..... 59
Kapitulli 8: Përfundime dhe Punë të Ardhshme ..... 61
8.1 Përmbledhje e Punimit ..... 61
8.2 Konkluzionet Teknike ..... 61
8.3 Punët e Ardhshme dhe Rekomandime ..... 62
8.4 Reflektim Final mbi Procesin e Punimit ..... 63
Bibliografia ..... 64
Shtojca A: Fragmente nga Kodi SystemVerilog i Komentuar ..... 65
Shtojca B: Lista e Plotë e Skedarëve të Projektit ..... 68
Shtojca C: Shpjegim i Detajuar Modul-për-Modul ..... 69

## LISTA E FIGURAVE

Figura 1: Pamja RTL e sintetizuar nga Vivado (Schematic/Elaborated Design). .....27
Figura 2: Verifikimi i parë i komunikimit serial UART përmes terminalit PuTTY.....39
Figura 3: Output-i përfundimtar i CoreMark në terminalin PuTTY, me validim të suksesshëm.....41

## LISTA E TABELAVE

Tabela 1: Klasifikimi i regjistrave 32-bit sipas konventës ABI në RISC-V. .....18
Tabela 2: Kodimi i sinjalit alu_control dhe burimi i tij nga fushat e instruksionit. .....19
Tabela 3: Përmbledhje e kostos në cikle fizike për secilin lloj instruksioni. .....32
Tabela 4: Harta e plotë e memories (Memory Map) e procesorit. .....38
Tabela 5: Rezultatet finale të ekzekutimit të CoreMark zyrtar EEMBC mbi FPGA real.....42
Tabela 6 Faktorët arkitekturorë që ndikojnë në diferencën e performancës me procesorë komercialë. .....43
Tabela 7 Përshkrimi i moduleve kryesore SystemVerilog të projektit. .....47
Tabela 8 Përmbledhje e sfidave kryesore, shtresës së sistemit të prekur, dhe mjetit të diagnostikimit. .....52
Tabela 9 Verifikimi i integritetit të kodit CoreMark zyrtar përmes checksum MD5. .....59
Tabela 10: Portet e modulit top.sv. .....69
Tabela 11: Portet e modulit riscv_cpu.sv. .....70
Tabela 12 Blloqet kryesore logjike të datapath.sv. .....71

## LISTE E DIAGRAMEVE

Diagrame 1  Bllokskema e sistemit top-level — lidhjet mes bërthamës CPU, memorieve të ndara (Harvard), dhe periferikëve memory-mapped (MMIO). .....21
Diagrame 2 Diagrami i detajuar i rrugës së të dhënave (datapath), me njësinë e kontrollit (Control Unit) në krye dhe multiplekserët kryesorë (pc_src, ALUSrc, Write-Back). .....23
Diagrame 3 Diagrami i gjendjeve (State Diagram) i FSM-së që sinkronizon CPU-në single-cycle me memorien BRAM sinkrone. .....25
Diagrame 4: Diagrami timing për një instruksion R-type/BRANCH — 2 cikle fizike (S_FETCH1 → S_FETCH2 → S_EXEC). .....30
Diagrame 5 Diagrami timing për një instruksion LOAD nga RAM — 4 cikle fizike, duke kaluar nëpër të gjitha 5 gjendjet e FSM-së. .....31
Diagrame 6 Diagrami timing për një instruksion STORE — 2 cikle fizike, pa nevojë për pritje konfirmimi nga BRAM. .....31
Diagrame 7 Procesi i plotë i kompilimit, nga kodi burimor C deri në bitstream-in fizik të FPGA-së. .....33
Diagrame 8 Sekuenca e zgjimit (boot sequence) e procesorit, e implementuar në start.S. .....35
Diagrame 9 Harta e plotë e hapësirës së adresave 32-bit të përdorur nga procesori. .....38
Diagrame 10 Rrjedha e thirrjeve të brendshme të CoreMark zyrtar EEMBC dhe rezultatet finale të matura. .....40
Diagrame 11 Hierarkia e instancimit të moduleve — top.sv është rrënja e gjithë sistemit të sintetizuar. .....47
Diagrame 12 Mekanizmi i bug-ut X-propagation dhe zgjidhja përmes zero-inicializimit të BRAM-it. .....54
Diagrame 13 Krahasimi i logjikës së gabuar (majtas) dhe të korrigjuar (djathtas) në next-state logic të FSM-së. .....55

## ABSTRAKTI

Ky punim diplome paraqet projektimin, implementimin, dhe validimin fizik të një procesori 32-bit të bazuar në arkitekturën e hapur RISC-V (RV32I), të ndërtuar tërësisht nga e para në gjuhën SystemVerilog dhe të vendosur mbi një platformë FPGA reale (Digilent Nexys A7-100T, Xilinx Artix-7). Procesori implementon një arkitekturë single-cycle me Harvard të modifikuar (memorie të ndara për kod dhe të dhëna), të kombinuar me një makinë gjendjesh (FSM) origjinale që sinkronizon modelin teorik single-cycle me kufizimet reale të memories Block RAM sinkrone të FPGA-së.

Për të vërtetuar funksionimin e procesorit në mënyrë objektive dhe shkencërisht të krahasueshme, u zhvillua një mjedis i plotë bare-metal (pa sistem operativ), që përfshin konfigurimin e toolchain-it RISC-V GCC, ndërtimin e një skedari Linker të personalizuar, implementimin e sekuencës së zgjimit (boot sequence), dhe portimin e benchmark-ut zyrtar industrial EEMBC CoreMark. Procesi i debug-imit zbuloi dhe zgjidhi disa probleme delikate karakteristike të sistemeve hardware-software bare-metal: propagim i vlerave të papërcaktuara (X-propagation) nga memorie e pa-inicializuar plotësisht, një gabim sinkronizimi "off-by-one cycle" në makinën e gjendjeve, çështje toolchain-i lidhur me Global Pointer dhe sekuencën e zgjimit, dhe një limit i brendshëm i mjetit të sintezës Vivado gjatë optimizimit të fuqisë.

Rezultati final i ekzekutimit të CoreMark zyrtar EEMBC tregoi përputhje bit-për-bit (CRC) me tabelat e publikuara zyrtarisht për konfigurimin "2K performance run", duke arritur një CoreMark Score prej 21.99 (Iterations/Sec) dhe një metrikë të normalizuar CoreMark/MHz prej 0.44 në frekuencën e funksionimit 50 MHz. Ky rezultat — i verifikuar përmes checksum MD5 kundrejt kodit burimor zyrtar EEMBC — përbën dëshmi të fortë të korrektësisë së plotë funksionale të arkitekturës RISC-V të implementuar, dhe ofron një bazë krahasimi direkte me procesorë komercialë të ngulitur.

## METODOLOGJIA E PËRDORUR

Metodologjia e këtij punimi është e strukturuar në katër faza të ndërvarura: dizajni RTL i hardware-it, ndërtimi i mjedisit softuerik bare-metal, sinkronizimi dhe debug-imi hardware-software, dhe validimi përmes benchmark-ut industrial.

Dizajni RTL. Arkitektura u projektua fillimisht në nivel konceptual (datapath, control unit, ALU) sipas modelit klasik single-cycle të përshkruar në literaturën standarde të arkitekturës së kompjuterave (Patterson & Hennessy), e implementuar në SystemVerilog dhe e verifikuar modul-për-modul përmes simulimit me Icarus Verilog përpara çdo sintezë mbi FPGA.

Mjedisi Softuerik Bare-Metal. U konfigurua toolchain-i xPack RISC-V GCC për kompilim cross-platform, u zhvillua një skedar Linker i personalizuar (linker.ld) dhe sekuenca e zgjimit (start.S), dhe u portua korniza CoreMark duke ofruar shtresën e nevojshme të ndërfaqes hardware (core_portme.c/.h).

Sinkronizimi dhe Debug-imi. Çdo problem i identifikuar — qoftë në simulim, qoftë mbi hardware real — u diagnostikua sipas një procesi sistematik: formulim hipoteze bazuar në simptomën e vrojtuar, izolim i problemit përmes testesh të dedikuara minimale, verifikim i zgjidhjes në simulim, dhe ri-konfirmim mbi FPGA fizik. Kjo metodologji u dokumentua në hollësi në Kapitullin 7.

Validimi. U përdor si standard validimi benchmark-u zyrtar EEMBC CoreMark, me kodin burimor algoritmik (core_list_join.c, core_matrix.c, core_state.c, core_util.c) të verifikuar checksum MD5 kundrejt publikimit zyrtar, për të garantuar që çdo krahasim CRC final të jetë shkencërisht i vlefshëm dhe i krahasueshëm me platforma të tjera.

# Kapitulli 1: Hyrje

## 1.1 Konteksti dhe Motivimi

Në dekadën e fundit, arkitektura RISC-V ka sjellë një ndryshim paradigme në industrinë e gjysmëpërçuesve dhe dizajnit të qarqeve të integruara. Në kontrast me arkitekturat tradicionale të pronësuara (si ARM apo x86), RISC-V ofron një set instruksionesh (ISA — Instruction Set Architecture) të hapur dhe pa kosto licencimi. Kjo qasje u mundëson inxhinierëve dhe studiuesve të dizajnojnë procesorë të personalizuar dhe të optimizuar për nevoja specifike kompjuterike, duke u çliruar nga kufizimet dhe kostot e licencimit komercial që karakterizojnë arkitekturat e mbyllura.

Ndërtimi i një procesori nga e para — duke filluar nga specifikimi i ISA-s, duke vazhduar me dizajnin RTL (Register-Transfer Level) në SystemVerilog, dhe duke përfunduar me verifikimin fizik mbi një platformë FPGA reale — përfaqëson një nga sfidat më komplekse dhe më formuese në fushën e Inxhinierisë Elektronike dhe të Arkitekturës së Kompjuterave. Ky proces detyron studentin të kapërcejë jo vetëm logjikën e nivelit të portave dhe regjistrave, por gjithashtu ndërveprimin delikate mes hardware-it dhe softuerit — që nga sinkronizimi i orës (clock) deri tek konfigurimi i një kompilatori cross-platform.

Ky projekt diplome lindi pikërisht nga nevoja për të eksploruar këto mundësi inxhinierike, duke ndërtuar nga e para një bërthamë procesori 32-bitëshe të aftë për të menaxhuar si instruksionet bazë aritmetike-logjike (RV32I), ashtu edhe zgjerimet e shumëzimit dhe pjesëtimit harduerik (RV32M). Zgjedhja e arkitekturës single-cycle si pikënisje u bë me qëllim pedagogjik të qartë: të kuptohet thellësisht çdo sinjal kontrolli përpara se të konsiderohen optimizime arkitekturore më komplekse si pipeline-i shumë-fazor.

## 1.2 Qëllimi i Punimit

Qëllimi kryesor i këtij punimi është dizajni në nivel RTL, implementimi dhe validimi fizik i një procesori RISC-V mbi një platformë FPGA reale (Digilent Nexys A7-100T, bazuar në çip-in Xilinx Artix-7). Përtej ndërtimit të vetë logjikës harduerike duke përdorur gjuhën SystemVerilog, ky punim synon të demonstrojë aftësinë e procesorit për të ekzekutuar korniza softuerike komplekse, jo thjesht programe testues të thjeshtë ("toy programs").

Për të vërtetuar këtë në mënyrë objektive dhe të krahasueshme shkencërisht, është zgjedhur standardi i industrisë EEMBC CoreMark si mjet matës i performancës. CoreMark është benchmark-u më i përdorur ndërkombëtarisht për procesorë të ngulitur (embedded), dhe ekzekutimi i tij i suksesshëm — me kontrollime CRC (Cyclic Redundancy Check) që përputhen ekzaktësisht me tabelat zyrtare të publikuara nga EEMBC — përbën provën më të fortë të mundshme që procesori ynë funksionon saktësisht sipas specifikimit RISC-V, pa asnjë devijim logjik që do të mund të kalonte pa u vënë re në teste më të thjeshta.

### 1.3 Problematika

Sfida kryesore e këtij projekti qëndron në sinkronizimin thelbësor midis dizajnit harduerik dhe përputhshmërisë me mjedisin softuerik (toolchain). Zhvillimi i një sistemi "bare-metal" (pa sistem operativ) kërkon një nivel shumë më të lartë inxhinierie sesa thjesht implementimi i instruksioneve aritmetike-logjike. Kjo kërkon hartimin e një sistemi të plotë I/O nga zeroja, menaxhimin e saktë të kohës (timer harduerik), dhe konfigurimin specifik të skedarëve të Linker-it për të përkthyer kodin e shkruar në gjuhën C në një ekzekutim binar të pastër, i cili duhet të vendoset saktësisht në adresat fizike të memories së FPGA-së.

Përveç kësaj, gjatë fazës praktike të implementimit u zbuluan probleme që nuk shfaqen kurrë në simulim të thjeshtuar: efekte si sinkronizimi i memories BRAM sinkrone me një CPU teorikisht "single-cycle", inicializimi i memories së pa-përdorur, dhe limite të vetë mjeteve të sintezës (Vivado) ndaj netlist-eve me kompleksitet të lartë. Këto probleme — dhe metodologjia e ndjekur për zgjidhjen e secilit — përbëjnë pjesën më të vlefshme dhe më origjinale të këtij punimi, dhe trajtohen në hollësi në Kapitullin 7.

### 1.4 Objektivat e Punimit

Për të përmbushur qëllimin kryesor, puna është ndarë në katër objektiva specifike, të renditura sipas varësisë logjike mes tyre:

- **Dizajni i Harduerit:** Implementimi i një arkitekture single-cycle 32-bit (RV32I, më vonë zgjeruar konceptualisht me M-extension) në gjuhën SystemVerilog, duke përfshirë rrugën e të dhënave (datapath), njësinë e kontrollit (decoder), memoriet e ndara të kodit

dhe të dhënave (arkitekturë Harvard e modifikuar), dhe periferikë memory-mapped (UART, timer harduerik).

- **Sinkronizimi Hardware-Memorie:** Ndërtimi i një makine gjendjesh (FSM) që përshtat dizajnin teorikisht single-cycle me kufizimet reale të memories BRAM sinkrone të FPGA-së, pa sakrifikuar korrektësinë funksionale të çdo instruksioni.
- **Ndërtimi i Toolchain-it:** Konfigurimi i kompilatorit RISC-V GCC (xPack distribution) për përpilimin e kodit C në një mjedis bare-metal, duke hequr varësitë nga libraritë standarde të sistemeve operative dhe duke ndërtuar nga zeroja sekuencën e zgjimit (boot sequence) të procesorit.
- **Validimi dhe Benchmark-u:** Portimi dhe ekzekutimi i suksesshëm i kornizës zyrtare EEMBC CoreMark, me qëllim arritjen e një rezultati CRC të vlefshëm zyrtarisht (jo thjesht ekzekutim pa krasha), dhe analiza krahasuese e performancës (CoreMark/MHz) kundrejt procesorëve komercialë të njohur.

### 1.5 Struktura e Punimit

Për të pasqyruar në mënyrë të qartë metodologjinë dhe rezultatet, ky punim është i ndarë si më poshtë:

- Kapitulli 2 trajton bazën teorike të arkitekturës RISC-V, formatet e instruksioneve, dhe konventën ABI të regjistrave.
- Kapitulli 3 detajon arkitekturën harduerike të ndërtuar në këtë projekt — datapath, ALU, njësia e kontrollit, dhe veçanërisht FSM-ja e sinkronizimit me memorien BRAM, e shoqëruar me diagrame të plota bllokskeme dhe gjendjesh.
- Kapitulli 4 përshkruan mjedisin softuerik, toolchain-in e kompilimit, skedarin Linker, sekuencën e zgjimit (boot sequence), dhe hartën e plotë të memories.
- Kapitulli 5 paraqet metodologjinë e testimit mbi FPGA real, ekzekutimin e CoreMark, dhe rezultatet finale të performancës, përfshirë krahasimin me procesorë komercialë.
- Kapitulli 6 shpjegon strukturën e kodit dhe organizimin e moduleve SystemVerilog, përfshirë hierarkinë e instancimit.

- Kapitulli 7 analizon në hollësi sfidat kryesore inxhinierike të hasura gjatë projektit dhe zgjidhjet teknike përkatëse — nga problemet e sinkronizimit fizik deri tek konfigurimi i toolchain-it dhe portimi i algoritmit zyrtar CoreMark.
- Kapitulli 8 përmbledh konkluzionet e punimit dhe ofron rekomandime për përmirësime arkitekturore të ardhshme.

## 1.6 Kontekst dhe Krahasim me Projekte të Ngjashme Open-Source

Implementimi i procesorëve RISC-V me burim të hapur (open-source) ka njohur një rritje të jashtëzakonshme në vitet e fundit, e nxitur kryesisht nga disponueshmëria e gjerë e specifikimit ISA dhe e mjeteve falas të sintezës/simulimit. Disa nga implementimet më të njohura akademike dhe industriale ofrojnë pikë referimi të dobishme për të kontekstualizuar zgjedhjet arkitekturore të këtij projekti:

- **PicoRV32 (Clifford Wolf):** Një bërthamë RV32IMC shumë e ngjeshur (compact), e optimizuar për madhësi minimale silikoni në vend të performancës maksimale — filozofi disi e ngjashme me këtë projekt, që gjithashtu prioritizoi thjeshtësinë dhe korrektësinë mbi optimizimin agresiv të performancës në fazën fillestare.
- **FemtoRV (Bruno Levy):** Familje procesorësh edukativë RV32I, me theks të veçantë në qartësinë e kodit për qëllime mësimore — disa nga skedarët fillestarë të portimit CoreMark në këtë projekt patën si referencë konceptuale strukturën e këtij projekti, përpara se të zëvendësoheshin me algoritmin zyrtar EEMBC (Kapitulli 7.6).
- **RISC-V Steel / Ibex (lowRISC):** Implementime më komplekse, me pipeline 2-3 fazor dhe mbështetje më të gjerë periferikësh, që përfaqësojnë "hapin tjetër" logjik të evolucionit arkitekturor të diskutuar në rekomandimet e Kapitullit 8.3.

Ndryshe nga shumë nga këto projekte, që shpesh raportojnë vetëm "ekzekutim i suksesshëm" ose teste sintetike të thjeshta, kontributi specifik i këtij punimi qëndron në dokumentimin e plotë dhe transparent të procesit të arritjes së një rezultati CoreMark zyrtarisht të vlefshëm — duke përfshirë çdo gabim hasur përgjatë rrugës (Kapitulli 7), jo vetëm rezultatin final të suksesshëm. Kjo qasje synon t'i shërbejë si referencë praktike studentëve të ardhshëm që do të ndërmarrin projekte të ngjashme.

# Kapitulli 2: Bazat Teorike dhe Arkitektura RISC-V

## 2.1 Çfarë është RISC-V?

RISC-V (lexohet "Risk-Five", duke përfaqësuar gjeneratën e pestë të procesorëve RISC të zhvilluar fillimisht në UC Berkeley) është një set instruksionesh (ISA — Instruction Set Architecture) i hapur dhe i lirë nga kosto licencimi. Për dallim nga arkitekturat e mbyllura dhe të pronësuara si x86 (Intel/AMD) apo ARM, RISC-V nuk kërkon licenca tregtare për t'u implementuar, studiuar, ose modifikuar.

Kjo natyrë open-source u mundëson inxhinierëve dhe studiuesve lirinë absolute për të analizuar, modifikuar, dhe dizajnuar procesorë të personalizuar. Filozofia themelore e RISC-V është thjeshtësia: specifikimi bazë nuk detyron implementimin e qindra instruksioneve të papërdorura (siç ndodh shpesh në ISA-të e vjetra CISC për arsye të përputhshmërisë historike), por lejon shtimin modular të zgjerimeve (extensions) — si M (Multiplication/Division), F (Floating Point), C (Compressed) — pa thyer përputhshmërinë me softuerin standard dhe kompilatorët e mjedisit. Për pajisjet me burime të kufizuara si FPGA-të, kjo do të thotë që një CPU mund të projektohet për të zënë hapësirën minimale të nevojshme (LUT-e, Flip-Flop-e), duke optimizuar konsumin e burimeve.

## 2.2 Arkitektura RV32I dhe Zgjerimi M

Arkitektura RISC-V është thellësisht modulare. Për këtë projekt është implementuar nënbashkësia bazë RV32I, e zgjeruar konceptualisht me zgjerimin M:

- **RV32I (Base Integer Set):** Seti bazë i instruksioneve 32-bit. Ky modul është thelbi i procesorit dhe përfshin rreth 40 instruksione esenciale për operacionet aritmetike (ADD, SUB), logjike (AND, OR, XOR), zhvendosjet (SLL, SRL, SRA), degëzimet e kushtëzuara (BEQ, BNE, BLT, BGE, BLTU, BGEU), kërcimet (JAL, JALR), dhe aksesin në memorie (LW, LB, LH, LBU, LHU, SW, SB, SH).
- **M (Multiplication/Division Extension):** Kjo shtesë mundëson operacione aritmetike të avancuara në harduer — specifikisht shumëzimin (MUL) dhe pjesëtimin (DIV, DIVU, REM, REMU). Implementimi harduerik i kësaj shtese rrit ndjeshëm performancën në algoritme me llogaritje intensive matricore, siç është pikërisht benchmark-u CoreMark. ALU-ja e këtij procesori i implementon të gjitha këto operacione (shih Kapitullin 3.3),

megjithëse — siç do të diskutohet në hollësi në Kapitullin 7.3 — gjatë fazës fillestare të implementimit u zgjodh të kompilohej softueri pa zgjerimin M (-march=rv32i), për të shmangur kufizime timing-u në divizorin kombinatorial.

Të gjitha instruksionet në arkitekturën RV32I kanë një gjatësi fikse prej 32 bitësh dhe ndahen në gjashtë formate kryesore, të cilat thjeshtojnë në maksimum logjikën e dekodimit në harduer:

- Formati R (Register): Për operacione midis dy regjistrave (p.sh. ADD rd, rs1, rs2).
- Formati I (Immediate): Për operacione midis një regjistri dhe një konstanteje 12-bit, si dhe për leximin nga memoria (p.sh. ADDI, LW).
- Formati S (Store): Përdoret ekskluzivisht për të ruajtur të dhëna nga një regjistër në memorie (p.sh. SW).
- Formati B (Branch): Përdoret për kërcime të kushtëzuara (p.sh. BEQ, BGE).
- Formati U (Upper Immediate): Për të ngarkuar konstante të mëdha (20-bit) në bitet e sipërme të regjistrave (p.sh. LUI).
- Formati J (Jump): Për kërcime të pakushtëzuara me rreze të gjatë (p.sh. JAL).

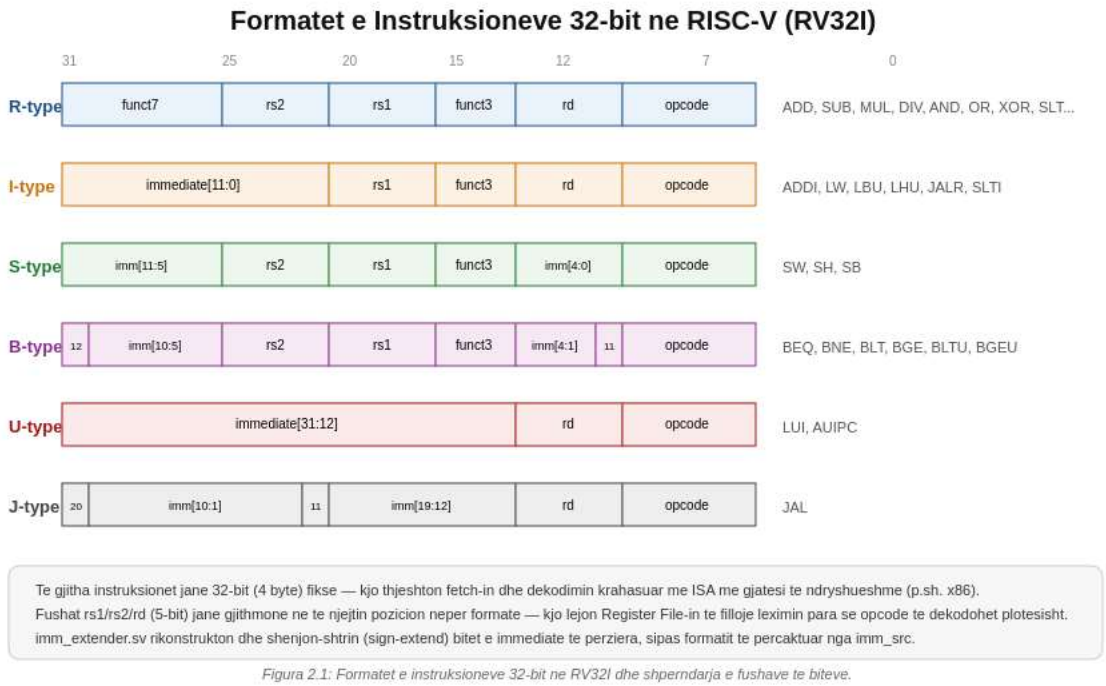



*Figura 2.1: Formatet e instruksioneve 32-bit ne RV32I dhe shperndarja e fushave te biteve.*

*Figure 1: Formatet e instruksioneve 32-bit në RV32I dhe shpërndarja e fushave të biteve.*

Vihet re që fushat rs1, rs2, dhe rd (secila 5-bit, që adresojnë 1 nga 32 regjistrat) qëndrojnë gjithmonë në të njëjtin pozicion bitesh nëpër formate të ndryshme — kjo është një zgjedhje e qëllimshme arkitekturore që lejon Register File-in (Kapitulli 3.2) të fillojë leximin e

operandeve paralelisht me dekodimin e plotë të opcode-it, duke kursyer kohë në rrugën kritike (critical path) të procesorit.

### 2.3 Arkitektura Load-Store dhe Ndërveprimi me Memorien

Procesori i dizajnuar ndjek rreptësisht arkitekturën Load-Store. Në arkitekturat CISC (si x86), një instruksion i vetëm mund të lexojë një vlerë nga memoria, ta përpunojë me një regjistër, dhe ta ruajë sërish në memorie — gjë që kërkon harduer tepër kompleks dhe cikle të paparashikueshme ore.

Në kontrast, në RISC-V, operacionet aritmetike-logjike (si mbledhja apo shumëzimi) lejohen të kryhen vetëm mbi të dhënat që tashmë ndodhen brenda regjistrave të procesorit. Aksesi në RAM bëhet ekskluzivisht përmes dy familjeve të instruksioneve: LOAD (LW, LH, LB, LHU, LBU) për të sjellë të dhëna nga memoria në regjistra, dhe STORE (SW, SH, SB) për të dërguar të dhëna nga regjistrat në memorie.

Kjo qasje, në një CPU teorikisht single-cycle, do të bënte që çdo instruksion aritmetik të ekzekutohej brenda një cikli të vetëm ore, duke lejuar një vlerësim të qartë dhe të parashikueshëm të performancës (CPI = 1, Cycles Per Instruction). Megjithatë, siç do të shihet në Kapitullin 3.5, implementimi real mbi FPGA me memorie BRAM sinkrone e detyroi këtë projekt të zgjerojë lehtësisht këtë model teorik përmes një makine gjendjesh (FSM) — pa humbur korrektësinë funksionale të arkitekturës Load-Store.

### 2.4 Regjistrat dhe Konventa e Thirrjes (ABI)

Zemra e përpunimit të të dhënave në RISC-V është Banka e Regjistrave (Register File). Arkitektura përdor 32 regjistra të përgjithshëm me gjerësi 32-bit (emërtuar nga x0 deri në x31). Regjistri x0 (Zero Register) është i lidhur fizikisht me vlerën konstante 0; ai nuk mund të mbishkruhet asnjëherë (shih implementimin në register_file.sv, Kapitulli 3.2) dhe përdoret masivisht për të sintetizuar instruksione të tjera (p.sh. lëvizja e një vlere realizohet si mbledhje me zero: MV rd, rs1 → ADDI rd, rs1, 0).

Për të lejuar që softueri i kompiluar në gjuhën C (si CoreMark) të komunikojë saktë me procesorin, regjistrave u janë caktuar role specifike nga Konventa e Ndërfaqes Binare-

Aplikacion (ABI). Përdorimi efikas i këtyre regjistrave minimizon thirrjet e ngadalta drejt memories RAM dhe garanton ekzekutimin optimal të funksioneve C.

| Regjistri Harduerik | Emri ABI | Përshkrimi dhe Përdorimi | Kush e Ruan? |
|---|---|---|---|
| x0 | zero | Konstante e hardkoduar 0. | N/A |
| x1 | ra | Return Address (adresa e kthimit pas thirrjes së një funksioni). | Caller |
| x2 | sp | Stack Pointer (tregon majën e Stack-ut në memorie). | Callee |
| x3 | gp | Global Pointer (tregon zonën e variablave globale — kritik për adresim GP-relative, shih Kap. 7.3). | N/A |
| x4 | tp | Thread Pointer (përdoret për sisteme multi-threading). | N/A |
| x5 - x7 | t0 - t2 | Regjistra të përkohshëm (temporary). | Caller |
| x8 | s0 / fp | Regjistër i ruajtur (saved) / Frame Pointer. | Callee |
| x9 | s1 | Regjistër i ruajtur (saved). | Callee |
| x10 - x11 | a0 - a1 | Argumentet e funksionit dhe vlerat e kthimit. | Caller |
| x12 - x17 | a2 - a7 | Argumentet e funksionit. | Caller |
| x18 - x27 | s2 - s11 | Regjistra të ruajtur shtesë. | Callee |
| x28 - x31 | t3 - t6 | Regjistra të përkohshëm shtesë. | Caller |

*Tabela 1: Klasifikimi i regjistrave 32-bit sipas konventës ABI në RISC-V.*

Njohja dhe mapimi i saktë i kësaj tabele nga ana e kompilatorit GCC është thelbësore për mjedise bare-metal si ky projekt. Për shembull, gjatë sekuencës së zgjimit të procesorit (boot sequence, shih Kapitullin 4.3), instruksioni i parë real i ekzekutuar vendos regjistrin x2 (sp) drejt fundit të hapësirës RAM (0x10000), duke lejuar që programet e shkruara në C — të cilat presupozojnë automatikisht ekzistencën e një Stack-u funksional — të mos krashtojnë procesorin gjatë thirrjes së parë të një funksioni.

### 2.5 Kodimi i Sinjalit alu_control

Decoder-i (riscv_isa.sv, i përshkruar në hollësi në Kapitullin 3.4) prodhon një sinjal 5-bit, alu_control, që drejton ALU-në drejt operacionit korrekt. Ky kodim është krijuar specifikisht për këtë implementim — jo standard i vetë RISC-V ISA — dhe demonstron se si dekodimi opcode/funct3/funct7 shndërrohet në një kod kontrolli kompakt të kuptueshëm nga harduer kombinatorial:

| alu_control | Operacioni | Burimi (funct3/funct7) |
|---|---|---|
| 5'b00000 | ADD | R-type funct3=000, funct7≠0100000/0000001 |
| 5'b01000 | SUB | R-type funct3=000, funct7[5]=1 |
| 5'b01001 | MUL | R-type funct3=000, funct7=0000001 (M-ext) |
| 5'b00001 | SLL | R/I-type funct3=001 |
| 5'b00010 | SLT | R/I-type funct3=010 |
| 5'b00011 | SLTU | R/I-type funct3=011 |
| 5'b00100 | XOR / DIV | funct3=100 (DIV nëse funct7=0000001) |
| 5'b00101 | SRL | R/I-type funct3=101, instr[30]=0 |
| 5'b01101 | SRA | R/I-type funct3=101, instr[30]=1 |
| 5'b00110 | OR / REM | funct3=110 (REM nëse funct7=0000001) |
| 5'b00111 | AND / REMU | funct3=111 (REMU nëse funct7=0000001) |
| 5'b10000-10011 | DIVU / DIV / REM / REMU | R-type, funct7=0000001 (M-ext, shih Kap. 3.3) |

*Tabela 2: Kodimi i sinjalit alu_control dhe burimi i tij nga fushat e instruksionit.*

Vihet re se i njëjti opcode bazë (R-type, 0110011) prodhon kode alu_control krejt të ndryshme varësisht funct7 — pikërisht ky mekanizëm lejon implementimin e të gjithë familjes ADD/SUB/MUL brenda të njëjtit "degëzim" kryesor të decoder-it, duke minimizuar numrin e krahasimeve logjike të nevojshme.

## 2.6 Shembull i Plotë: Dekodimi Hap-pas-Hapi i një Instruksioni

Për të ilustruar konkretisht si funksionon zinxhiri i plotë fetch-decode-execute, le të ndjekim instruksionin LBU x12, 0(x10) — pikërisht instruksioni që luajti rol qendror në diagnostikimin e bug-ut të Kapitullit 7.2. Në kod heksadecimal, ky instruksion shfaqet si 0x00054603 brenda memfile.dat.

Instruksioni (binar): 0000 0000 0000 0101 0100 0110 0000 0011
|funct7 | rs2 | rs1 |fn3| rd |opcode |

Fushat e nxjerra:
opcode = 0000011 (binar) = 0x03 -> LOAD (Kapitulli 3.4)
funct3 = 100 -> LBU specifikisht (jo LB/LH/LHU/LW)
rs1 = 01010 = x10 -> baza e adresimit
rd = 01100 = x12 -> regjistri destinacion
imm[11:0] = 000000000000 -> offset = 0

Brenda riscv_isa.sv, opcode=0000011 aktivizon degën LOAD: reg_write=1, alu_src=1 (operandi i dytë i ALU është immediate, jo regjistër), mem_to_reg=1 (rezultati vjen nga memoria, jo nga ALU), dhe is_lbu=1 (funct3=100 specifikisht). Brenda datapath.sv, ALU llogarit alu_result = rs1 + imm = x10 + 0 — kjo është adresa e plotë e bajtit për t'u lexuar.

Kjo adresë (në shembullin real të debug-imit, 0x40cc) dërgohet drejt FSM-së në top.sv (Kapitulli 3.5), e cila — meqenëse instruksioni është LOAD nga RAM e vërtetë (adresa < 0x10000) — kalon nëpër gjendjet S_DECODE_LOAD dhe S_LOAD_WAIT për t'i dhënë BRAM-it kohën e nevojshme. Pasi data_mem kthen fjalën e plotë 32-bit që përmban bajtin e kërkuar, datapath.sv (blloku "aligner" i përshkruar në Kapitullin 3.6) ekstrakton bajtin specifik bazuar te alu_result[1:0], dhe e zero-shtrin (sepse është LBU, jo LB) në 32-bit të plota përpara se ta shkruajë në regjistrin x12.

Ky shembull konkret demonstron pse korrektësia e çdo shtrese — decoder, ALU, FSM, dhe aligner — duhet të jetë e përsosur njëkohësisht: një gabim në ndonjë nga këto katër hapa (siç ndodhi fillimisht me FSM-në, Kapitulli 7.2) do të prodhonte një vlerë të gabuar në x12, që më pas do të përhapej në çdo llogaritje të mëvonshme që e përdor këtë regjistër.

# Kapitulli 3: Arkitektura dhe Dizajni i Sistemit Hardware

## 3.1 Përmbledhje e Arkitekturës së Sistemit

Sistemi i dizajnuar është një procesor 32-bit i bazuar në arkitekturën RISC-V (RV32I, e zgjeruar konceptualisht me M). Për të optimizuar shpejtësinë e aksesit në të dhëna dhe instruksione, procesori implementon një Arkitekturë Harvard të Modifikuar: dizajni është ndarë në blloqe të pavarura fizikisht — Instruction Memory (instr_mem.sv, memoria e kodit) dhe Data Memory (data_mem.sv, memoria e të dhënave). Këto dy memorie ndërveprojnë me bërthamën e procesorit (riscv_cpu) përmes dy bus-eve të ndara fizikisht, duke lejuar leximin e një instruksioni dhe leximin/shkrimin e një të dhëne brenda të njëjtit "instruksion logjik", pa konflikt burimi.

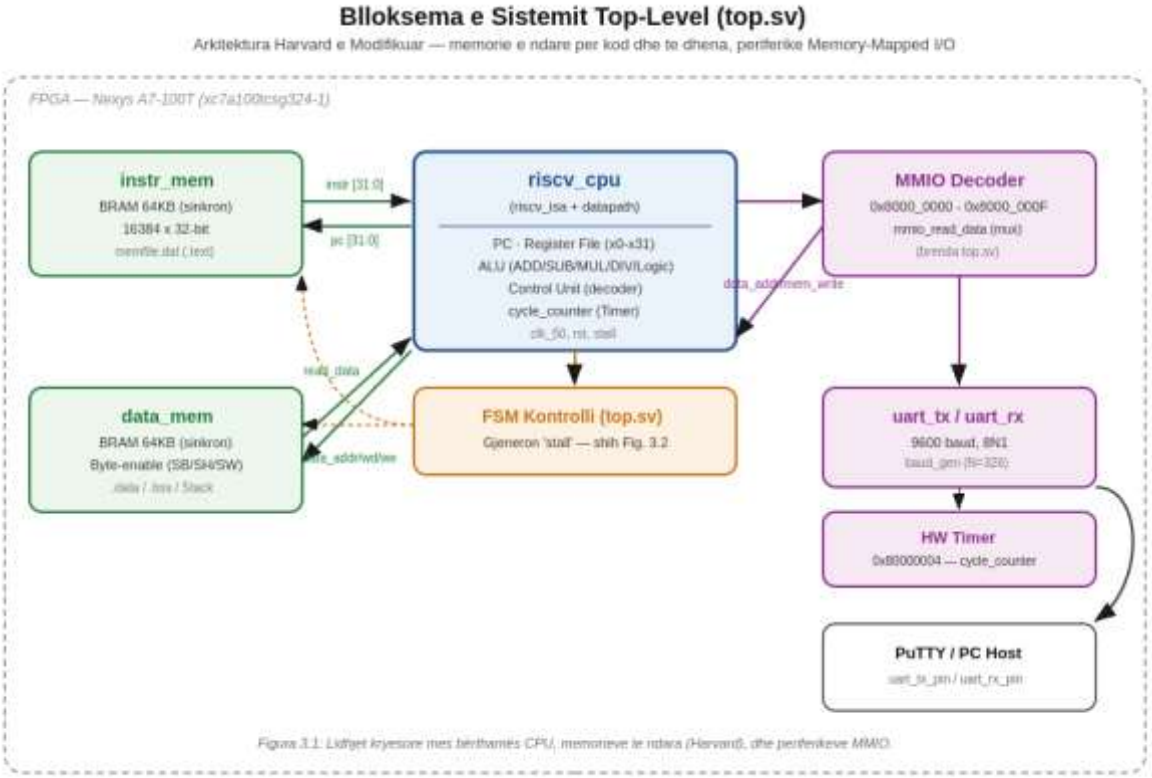



*Figura 3.1: Lidhjet kryesore mes bërthamës CPU, memorieve të ndara (Harvard), dhe periferikeve MMIO.*

*Diagrame 1 Bllokskema e sistemit top-level — lidhjet mes bërthamës CPU, memorieve të ndara (Harvard), dhe periferikëve memory-mapped (MMIO).*

Siç ilustrohet në Figurën 3.1, moduli top.sv shërben si "integrues" qendror: instancion riscv_cpu (bërthama e procesorit), instr_mem dhe data_mem (memoriet), uart_tx/uart_rx/baud_gen (komunikimi serial), dhe një makinë gjendjesh (FSM) që gjeneron

sinjalin kritik stall — mekanizmi që mundëson sinkronizimin mes një CPU-je teorikisht single-cycle dhe memories BRAM reale, sinkrone, që kërkon 1 cikël vonesë për çdo lexim (shih Kapitullin 3.5).

## 3.2 Rruga e të Dhënave (Datapath) dhe Dizajni Single-Cycle

Bërthama e procesorit (riscv_cpu.sv) është implementuar duke përdorur një arkitekturë single-cycle: çdo instruksion i programit përpunohet i plotë, nga fetch deri tek write-back, brenda një "cikli logjik" të vetëm. Rruga e të dhënave (datapath.sv) integron komponentët e mëposhtëm në një rrjedhë logjike të pandërprerë:

- **Program Counter (PC):** Regjistri kryesor sinkron që mban adresën aktuale të instruksionit. Përditësohet në çdo teh rritës (posedge) të orës, duke kaluar në PC+4 (ekzekutim sekuencial) ose në një adresë të re gjatë instruksioneve të degëzimit/kërcimit (Branch/JAL/JALR).
- **Banka e Regjistrave (Register File):** Një njësi memorieje shumë e shpejtë që përmban 32 regjistra 32-bit (x0-x31). Ka dy porta asinkrone leximi (për të marrë operandet SrcA dhe SrcB njëkohësisht) dhe një portë sinkrone shkrimi, e kushtëzuar nga sinjali RegWrite & ~stall.
- **Zgjeruesi i Vlerave (Immediate Extender):** Nxjerr vlerat konstante nga bitet e instruksionit (sipas formatit I/S/B/J/U, shih Kapitullin 2.2) dhe i zgjeron ato në 32-bit duke ruajtur saktë bitin e shenjës (sign extension).

### 3.2.1 Karakteristikat e Kohës (Timing) të Register File

Register File-i (register_file.sv) ka një karakteristikë timing-u që meriton vëmendje të veçantë: leximi është ASINKRON (kombinatorial, pa always_ff), ndërsa shkrimi është SINKRON (i regjistruar, brenda always_ff @(posedge clk)). Kjo kombinim — standarde në shumicën e implementimeve RISC-V akademike — lejon që, brenda të NJËJTIT cikël S_EXEC, datapath.sv të lexojë operandet (src_a, src_b_wire) DHE të shkruajë rezultatin e ri (result) në regjistrin destinacion, pa konflikt, sepse leximi 'sheh' vlerën E VJETËR (përpara shkrimit aktual, që ndodh vetëm në fund të ciklit, në posedge tjetër).

Kjo do të thotë, konkretisht: nëse një instruksion është ADD x5, x5, x5 (rs1=rs2=rd=x5), ALU-ja merr vlerën E VJETËR të x5 (dy herë, për të dyja operandet), dhe rezultati i ri shkruhet vetëm në FUND të ciklit clock — pa pasur nevojë për ndonjë mekanizëm shtesë "bypass" apo "forwarding", siç do të kërkohej në një pipeline multi-fazor (shih Kapitullin 3.8). Kjo sjellje është pikërisht arsyeja pse register file-t single-cycle mund të jenë kaq të thjeshtë krahasuar me ato të një pipeline real.

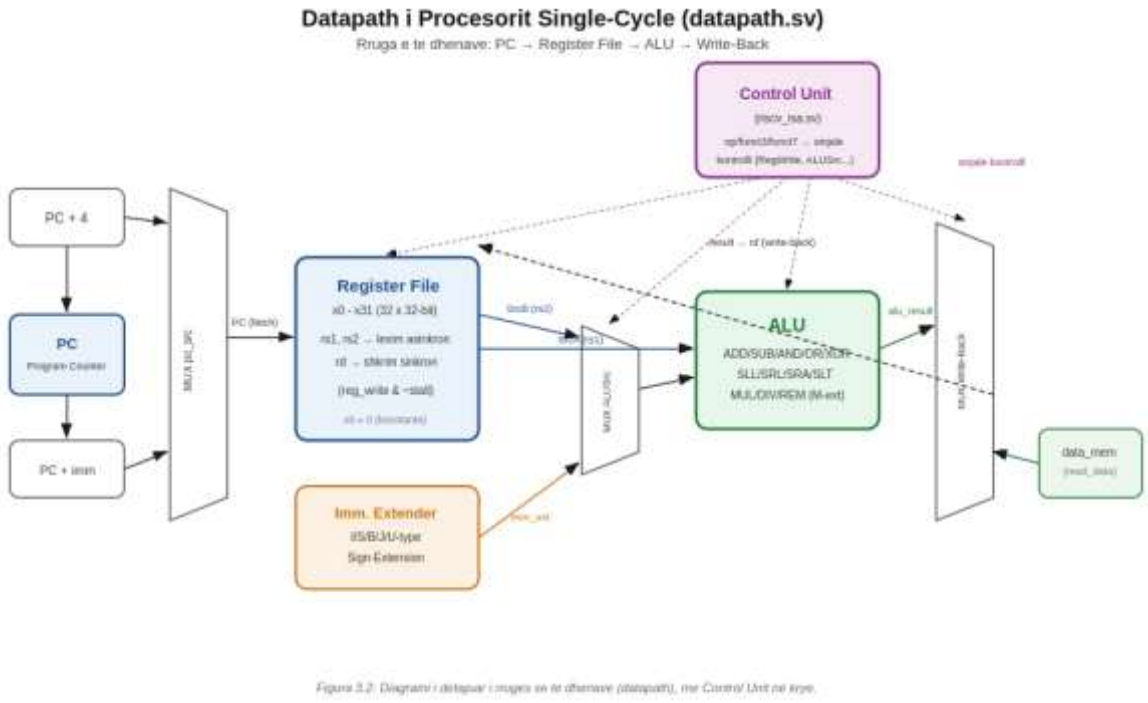



*Figura 3.2: Diagrami i detajuar i rrugës së të dhënave (datapath), me Control Unit në krye.*

*Diagrame 2 Diagrami i detajuar i rrugës së të dhënave (datapath), me njësinë e kontrollit (Control Unit) në krye dhe multiplekserët kryesorë (pc_src, ALUSrc, Write-Back).*

Diagrami i Figurës 3.2 tregon tre multiplekserë kyç që përcaktojnë sjelljen e procesorit për çdo lloj instruksioni: (1) MUX pc_src zgjedh mes PC+4 (ekzekutim normal) dhe pc_target (degëzim/kërcim i suksesshëm); (2) MUX ALUSrc zgjedh nëse operandi i dytë i ALU-së vjen nga regjistri rs2 ose nga vlera immediate; (3) MUX Write-Back zgjedh nëse vlera për të shkruar në regjistrin rd vjen nga ALU, nga memoria (për LOAD), nga PC+4 (për JAL/JALR), ose direkt nga immediate (për LUI).

### 3.3 Njësia Aritmetike-Logjike (ALU)

ALU (Arithmetic Logic Unit, alu.sv) është "truri" matematikor i procesorit. Është implementuar si një bllok logjik tërësisht kombinatorial (pure combinational logic, pa asnjë always_ff) që merr dy operande 32-bit dhe një sinjal kontrolli 5-bit (alu_control) nga Control Unit, për të gjeneruar një rezultat të menjëhershëm. ALU-ja e këtij projekti mbështet plotësisht operacionet e kërkuara nga standardi RV32I dhe shtesa M:

- Operacione Aritmetike: Mbledhje (ADD) dhe Zbritje (SUB).
- Operacione Logjike: AND, OR, XOR.
- Zhvendosje të Biteve (Shifts): Logjike Majtas/Djathtas (SLL, SRL), dhe Aritmetike Djathtas (SRA, që përdor operatorin e ruajtjes së shenjës $signed(a) >>> b[4:0] në SystemVerilog).
- Krahasime: SLT (Set Less Than, signed) dhe SLTU (unsigned), të përdorura si nga instruksionet SLTI/SLTIU ashtu edhe nga degëzimet BLT/BGE/BLTU/BGEU.
- Zgjerimi M: MUL (shumëzim), DIV/DIVU (pjesëtim signed/unsigned), REM/REMU (mbetja e pjesëtimit) — secili i mbrojtur kundër pjesëtimit me zero sipas specifikimit RISC-V (DIV/DIVU me b=0 kthen gjithë-1; REM/REMU me b=0 kthen vetë operandin a).

### 3.4 Njësia e Kontrollit (Control Unit)

Ndryshe nga arkitekturat multi-cycle që kërkojnë një makinë gjendjesh (FSM) komplekse për të drejtuar vetë ekzekutimin e çdo instruksioni, Njësia e Kontrollit (riscv_isa.sv) në këtë dizajn është tërësisht kombinatoriale. Ajo lexon fushat opcode (7-bit), funct3 (3-bit), dhe funct7 (7-bit) të instruksionit, dhe ngre menjëherë "flamujt" e kontrollit (control signals) për të hapur ose mbyllur rrugët e duhura të të dhënave brenda datapath-it. Sinjalet kyçe përfshijnë:

- **RegWrite:** Lejon shkrimin e rezultatit në Register File.
- **MemWrite:** Aktivizon portën e shkrimit në Data Memory (për instruksionet STORE).
- **ALUSrc:** Zgjedh nëse ALU merr operandin e dytë nga regjistri rs2 apo nga vlera immediate.
- **MemToReg:** Zgjedh nëse regjistri destinacion merr të dhëna nga ALU, nga Data Memory (LOAD), apo nga një kalkulim mbi PC (si në rastin e JAL/JALR/AUIPC).

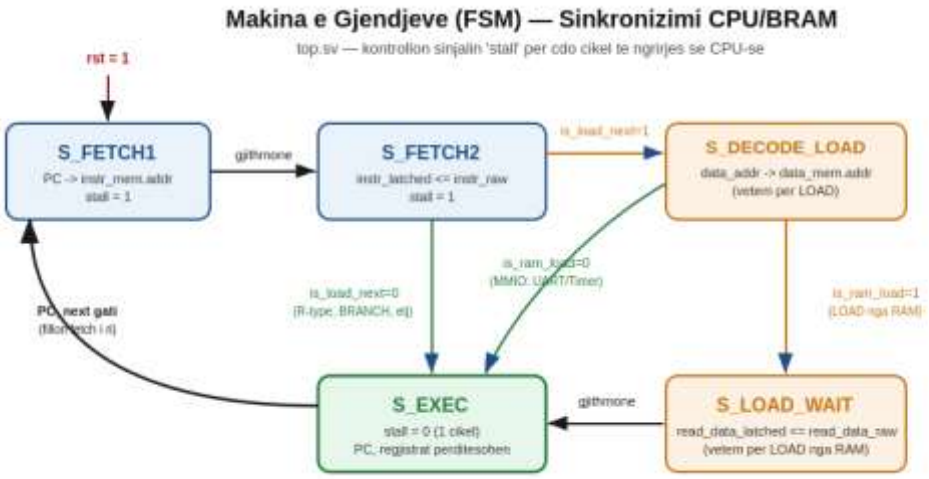
- **Branch / JAL / JALR:** Identifikojnë llojin specifik të kërcimit/degëzimit, që pastaj kombinohet me rezultatin e ALU-së (zero, cmp_result) për të vendosur nëse PC duhet të ndryshojë drejtimin normal të ekzekutimit.

### 3.5 Sinkronizimi me Memorien BRAM dhe Makina e Gjendjeve (FSM)

Ky është seksioni qendror i kontributit teknik origjinal të këtij projekti. Një CPU single-cycle "i pastër" presupozon që memoria e instruksioneve dhe ajo e të dhënave janë lexuar në mënyrë kombinatoriale — pra brenda të njëjtit cikël ore, adresa e dërguar prodhon menjëherë rezultatin, pa vonesë. Kjo është e vlefshme për memorie të vogla të bazuara në LUT (distributed RAM), por bëhet impraktike për memorie të mëdha (64 KB në këtë projekt), të cilat duhet të sintetizohen si Block RAM (BRAM) — burime hardware të dedikuara të FPGA-së.

Problemi themelor: BRAM-i është sinkron — adresa e dërguar në ciklin N prodhon të dhënën korrekte vetëm në fillim të ciklit N+1 (1 cikël vonesë). Nëse CPU-ja do të vazhdonte si single-cycle "i pastër" duke ia kaluar këtë të dhënë ALU-së në të njëjtin cikël, do të merrte një vlerë të papërcaktuar (e cila ekziston ende nga cikli i mëparshëm), duke korruptuar plotësisht ekzekutimin.

Zgjidhja e zbatuar në këtë projekt është një makinë gjendjesh (FSM) e vogël, e implementuar brenda top.sv, që "shtrin" çdo instruksion logjik në 2 deri në 4 cikle fizike clock, varësisht nga lloji i instruksionit, dhe gjeneron sinjalin stall që "ngrin" PC-në dhe regjistrat e procesorit derisa memoria të jetë gati.




*Diagrame 3 Diagrami i gjendjeve (State Diagram) i FSM-së që sinkronizon CPU-në single-cycle me memorien BRAM sinkrone.*

Pesë gjendjet e FSM-së funksionojnë si më poshtë:

- **S_FETCH1:** PC-ja aktuale dërgohet te instr_mem.addr. BRAM-i fillon "të llogarisë" instruksionin përkatës (proces që zgjat 1 cikël në hardware sinkron).
- **S_FETCH2:** Instruksioni (tani i vlefshëm te instr_raw) kapet në regjistrin instr_latched, që mbetet i palëvizshëm gjatë gjithë pjesës tjetër të ekzekutimit të këtij instruksioni. Nëse instruksioni është LOAD, FSM-ja kalon te S_DECODE_LOAD; përndryshe (R-type, BRANCH, STORE, etj.) kalon direkt te S_EXEC.
- **S_DECODE_LOAD:** Vetëm për instruksionet LOAD. Tani që instr_latched është i vendosur, ALU-ja brenda riscv_cpu ka llogaritur saktë adresën e leximit (data_addr). Nëse kjo adresë bie brenda RAM-it real, FSM-ja kalon te S_LOAD_WAIT; nëse bie në hapësirën MMIO (UART/Timer), e dhëna është gati menjëherë (lexim kombinatorial, jo BRAM), dhe FSM-ja kapërcen direkt te S_EXEC.
- **S_LOAD_WAIT:** 1 cikël pritjeje shtesë, i nevojshëm vetëm për LOAD nga RAM-i real, derisa BRAM-i (data_mem) të kthejë të dhënën korrekte.
- **S_EXEC:** I vetmi cikël në të cilin stall=0. Datapath-i kompleton write-back-un (shkrimin në regjistër), llogarit PC-në e ardhshme, dhe nëse instruksioni është STORE, aktivizon shkrimin real në memorie. Në fund të këtij cikli, PC-ja ndryshon dhe FSM-ja kthehet te S_FETCH1 për instruksionin tjetër.

Kosto mesatare e këtij dizajni është rreth 2 cikle fizike për instruksion (jo-LOAD) dhe rreth 4 cikle për instruksionet LOAD nga RAM — krahasuar me 1 cikël teorik të një single-cycle ideal pa kufizime memorie. Ky kompromis (throughput më i ulët, por korrektësi e garantuar) është diskutuar më gjerësisht në Kapitullin 7.2, ku përshkruhet edhe një gabim specifik ("off-by-one cycle") i zbuluar dhe i korrigjuar gjatë fazës së debug-imit.

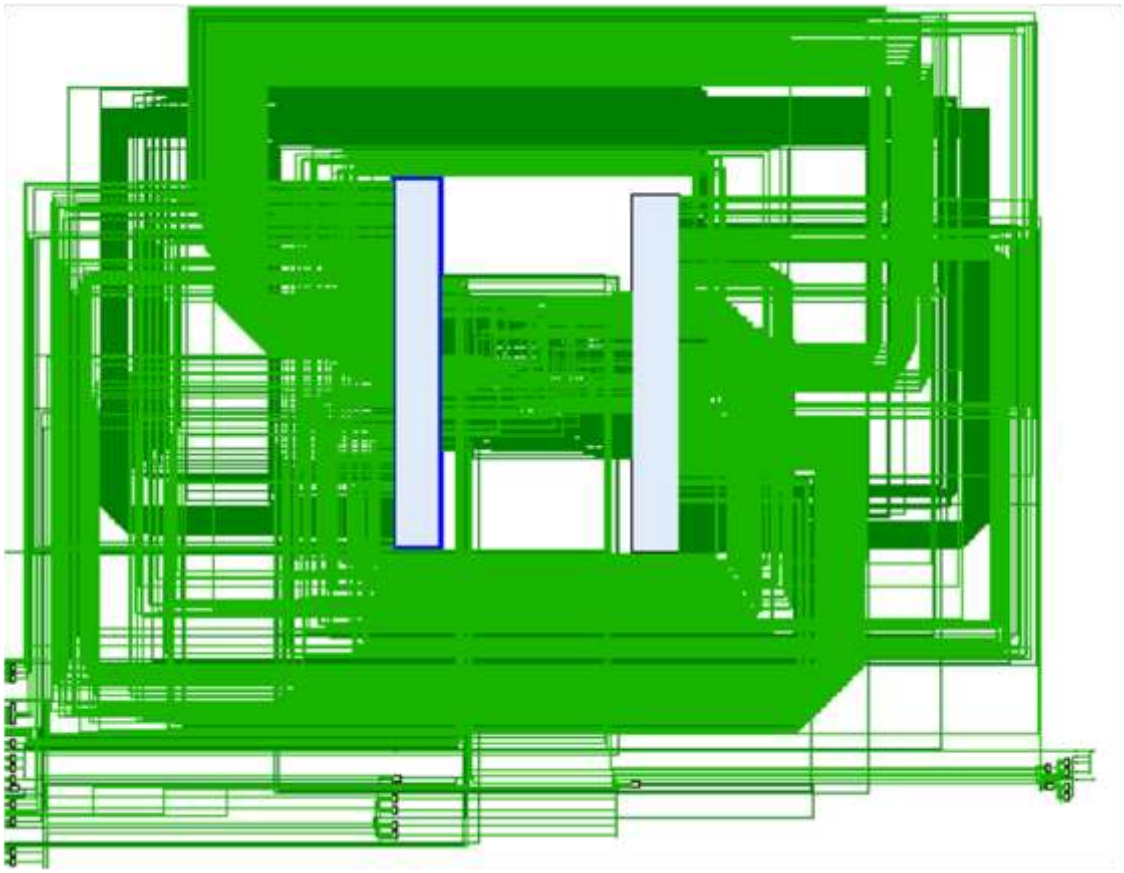


*Figura 1: Pamja RTL e sintetizuar nga Vivado (Schematic/Elaborated Design).*

### 3.6 Ndërfaqja e Memories dhe Instruksioni LBU

Memoria e të dhënave (data_mem.sv) është ndërtuar duke përdorur burimin Block RAM (BRAM) të FPGA-së, me logjikë byte-enable për të mbështetur instruksionet STORE të madhësive të ndryshme (SB, SH, SW). Një sfidë teknike specifike ishte manipulimi i të dhënave në nivel bajti (8-bit): kompilatori GCC, veçanërisht gjatë ekzekutimit të algoritmeve si State Machine në CoreMark, përdor masivisht instruksionin LBU (Load Byte Unsigned) për të lexuar karaktere individuale nga stringjet e ruajtura në memorie.

Për ta mbështetur këtë në harduer, datapath.sv implementon një logjikë "aligner" për leximet nga memoria: kur Control Unit detekton një LBU, procesori lexon të gjithë fjalën 32-bit nga adresa e RAM-it (sepse BRAM-i punon në granularitet fjale), ekstrakton vetëm 8 bitët e kërkuar (bazuar në 2 bitet e fundit të adresës, alu_result[1:0]), dhe i mbush 24 bitët e tjerë me zero (zero-extension) përpara se t'i dërgojë në Register File. Po ashtu implementohen LB (sign-extension për byte), LH/LHU (analoge për half-word). Ky mekanizëm — i thjeshtë në koncept

por delikat në implementim — ishte kufiri ndarës midis një sistemi që krashtonte dhe një sistemi të qëndrueshëm.

### 3.7 Sistemi Memory-Mapped I/O (MMIO) dhe Periferikët

Për ta lidhur bërthamën RISC-V me botën e jashtme, u përdor teknika e ndarjes së hartës së memories (Memory-Mapped I/O). Brenda top.sv, një dekoder kombinatorial (mmio_read_data) përgjon adresat e kërkuara nga CPU. Nëse adresa bie në zonën e RAM-it (< 0x00010000), të dhënat lexohen/shkruhen aty përmes data_mem. Nëse adresa bie në hapësirën e rezervuar 0x80000000–0x8000000F, hardueri aktivizon modulet përkatëse periferike:

- **UART Data (0x80000000):** Shkrimi i një bajti këtu aktivizon transmetimin serial drejt terminalit (p.sh. PuTTY). Moduli UART funksionon në frekuencën clk_50 (50 MHz) me standardin 9600 baud, 8 bit të dhëna, 1 stop bit (8N1).
- **Timer Harduerik (0x80000004):** Një regjistër 32-bit (cycle_counter, brenda riscv_cpu.sv) që rritet vazhdimisht me çdo cikël ore, pavarësisht nga stall. Ky modul siguron matjen absolute të kohës së kaluar, duke i mundësuar CoreMark-ut të llogarisë saktësisht numrin e "ticks" dhe performancën finale në Iterations/Sec (shih Kapitullin 5).
- **UART Status (0x80000008):** Lexim 1-bit që tregon nëse UART-i është i zënë (1) ose i lirë (0) për transmetim të ri. Softueri (uart_putchar) bën "spin-lock" mbi këtë regjistër për të shmangur korruptimin e të dhënave gjatë transmetimit (shih Kapitullin 4.4).

### 3.8 Pse Hazardet Klasike të Pipeline-it Nuk Aplikohen (dhe Çfarë Ndryshon nëse Shtohet Pipeline)

Një pyetje e natyrshme për dikë të njohur me arkitekturën e kompjuterave është: a vuan ky procesor nga hazardet klasike të pipeline-it (data hazards, control hazards, structural hazards)? Përgjigja e shkurtër është JO — dhe arsyeja është thelbësore për të kuptuar dallimin mes single-cycle (edhe me FSM shtesë për BRAM) dhe pipeline real.

Në një arkitekturë single-cycle — qoftë "e pastër" teorike, qoftë e zgjeruar me FSM si në këtë projekt — çdo instruksion përfundon PLOTËSISHT (fetch, decode, execute, memory access, write-back) përpara se instruksioni tjetër të fillojë fetch-in e tij. Kjo eliminon nga rrënja çdo

lloj hazard-i: nuk mund të ketë "data hazard" (instruksioni N+1 lexon një regjistër para se instruksioni N ta shkruajë), sepse instruksioni N ka përfunduar write-back-un e tij të plotë para se N+1 të fillojë fare; nuk mund të ketë "control hazard" (degëzim i papërcaktuar ende kur instruksioni tjetër fillon fetch), sepse PC-ja e ardhshme është gjithmonë e vendosur përfundimisht përpara se cikli tjetër logjik të fillojë.

Kostoja e kësaj thjeshtësie është throughput-i: siç u diskutua në Kapitullin 3.5, çdo instruksion kërkon 2-4 cikle FIZIKE clock (jo 1, siç do të ishte në single-cycle "ideal" pa BRAM), dhe asnjë instruksion nuk mund të "mbivendoset" me tjetrin. Nëse procesori do të evoluonte drejt një pipeline real (rekomandimi i Kapitullit 8.3), këto hazarde do të DUHEJ të adresoheshin shprehimisht: do të nevojitej forwarding logic (për data hazards, kur një instruksion në fazën Execute ka nevojë për një rezultat ende "në fluturim" nga instruksioni i mëparshëm), dhe një mekanizëm flush/stall (për control hazards, kur një degëzim rezulton i ndryshëm nga sa ishte parashikuar). Mungesa aktuale e këtyre mekanizmave NUK është një mangësi e këtij projekti — është thjesht karakteristikë e drejtë e arkitekturës zgjedhur, dhe çdo evolucion i ardhshëm drejt pipeline duhet t'i trajtojë këto çështje si pjesë integrale e dizajnit të ri.

### 3.9 Harvard kundër Von Neumann: Implikimet Praktike të Zgjedhjes Arkitekturore

Zgjedhja e arkitekturës Harvard (memorie të ndara për kod dhe të dhëna, Kapitulli 3.1) në vend të Von Neumann (memorie e përbashkët) ka implikime praktike që shkuan përtej thjesht "performancës teorike", dhe që u zbuluan konkretisht gjatë portimit të toolchain-it (Kapitulli 7.3).

Në një sistem Von Neumann "klasik", linker script-i mund të supozojë se .text, .rodata, .data, dhe .bss ndajnë të njëjtën hapësirë lineare adresash fizike — çfarë programatori sheh në ELF-in e kompiluar korrespondon drejtpërdrejt me një memorie të vetme fizike. Në Harvard, kjo supozim THYHET implicit: linker.ld i këtij projekti (Kapitulli 4.2) vendos .text në një hapësirë "virtuale" (0x00000000+) që në realitet harduerik korrespondon me instr_mem, ndërsa .data/.bss (po ashtu brenda të njëjtit hapësirë lineare adresash, sipas linker script-it) korrespondojnë në fakt me një memorie FIZIKE krejt të ndarë (data_mem).

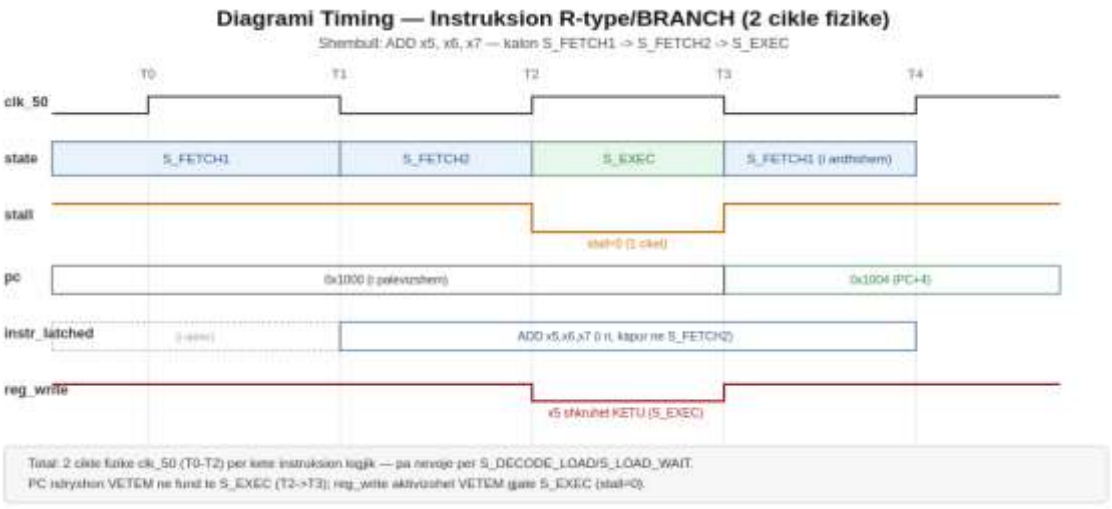
Kjo "iluzion" funksionon pa probleme PËR SA KOHË që firmware-i kurrë nuk përpiqet të lexojë .data/.bss përmes një instruksioni FETCH (sepse instr_mem nuk i përmban ato të dhëna fare), ose të ekzekutojë kod të ruajtur brenda .data (sepse instr_mem është memoria e VETME

nga e cila CPU bën fetch). Për kod "normal" C (jo kod vetë-modifikues, jo JIT compilation), ky kufizim nuk paraqet problem real — por është një supozim i fshehur që programatori i sistemeve bare-metal duhet ta kuptojë EXPLICITE, sepse asnjë mesazh gabimi nuk do ta sinjalizonte automatikisht nëse do të shkelej (CPU thjesht do të lexonte të dhëna të papërcaktuara nga instr_mem, duke prodhuar sjellje të gabuar të qetë, e jo një "crash" të dukshëm).

### 3.10 Diagramet Timing: Ekzekutimi Cikël-pas-Cikli

Për të plotësuar diagramin abstrakt të gjendjeve (Figura 3.3) me një pamje konkrete të sjelljes në kohë, kjo seksion paraqet diagramet timing (waveform) për tre lloje përfaqësuese instruksionesh: R-type/BRANCH (më i shpejti, 2 cikle), LOAD nga RAM (më i ngadalshmi, 4 cikle), dhe STORE (2 cikle, por me sjellje "fire-and-forget" të ndryshme nga R-type).

#### 3.10.1 R-type / BRANCH (2 Cikle)

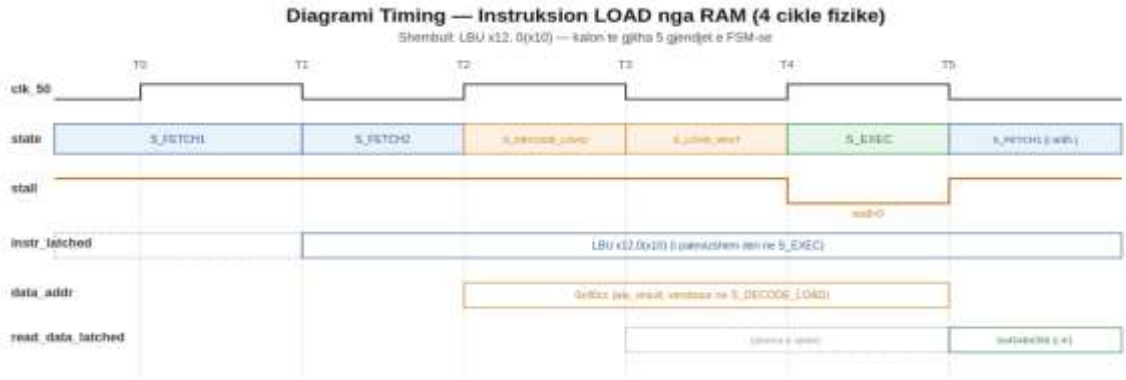



*Diagrame 4: Diagrami timing për një instruksion R-type/BRANCH — 2 cikle fizike (S_FETCH1 → S_FETCH2 → S_EXEC).*

Ky është rasti më i shpejtë i mundshëm: instruksioni kalon vetëm përmes S_FETCH1 (adresimi i instr_mem) dhe S_FETCH2 (kapja e instruksionit), përpara se të arrijë direkt te S_EXEC, ku stall=0 lejon PC-në dhe regjistrat të përditësohen. Kjo familje përfshin të gjitha instruksionet R-type (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU, MUL, DIV, etj.), I-type

ALU (ADDI, ANDI, etj.), dhe BRANCH (BEQ, BNE, BLT, etj.) — pra shumicën dërrmuese të instruksioneve në një program tipik C.

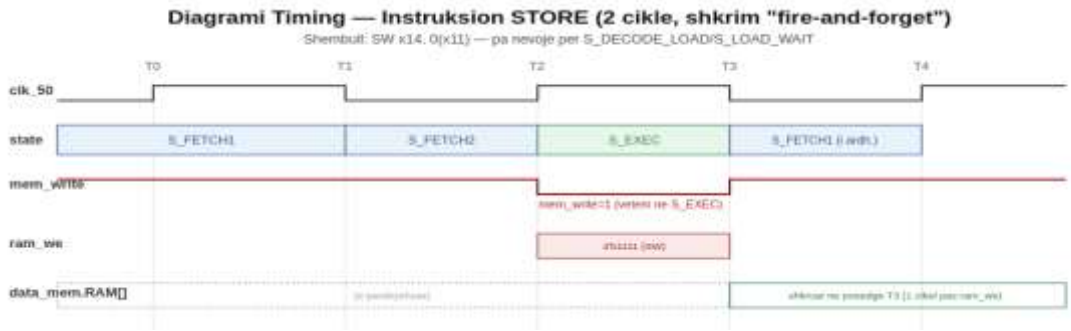
### 3.10.2 LOAD nga RAM (4 Cikle)




*Diagrame 5 Diagrami timing për një instruksion LOAD nga RAM — 4 cikle fizike, duke kaluar nëpër të gjitha 5 gjendjet e FSM-së.*

Rasti më i shtrenjtë: LOAD nga RAM e vërtetë (jo MMIO) duhet të kalojë nëpër S_DECODE_LOAD (ku ALU-ja llogarit data_addr, bazuar te instr_latched i tashmë i vendosur) dhe S_LOAD_WAIT (1 cikël pritjeje shtesë, i nevojshëm specifikisht sepse BRAM-i është sinkron dhe kthen të dhënën VETËM 1 cikël pas adresimit). Vihet re se data_addr bëhet i vlefshëm vetëm gjatë S_DECODE_LOAD (jo më herët), dhe read_data_latched merr vlerën korrekte vetëm në FUND të S_LOAD_WAIT — pikërisht ky moment timing-u ishte burimi i bug-ut "off-by-one cycle" të Kapitullit 7.2, kur next-state logic-u gabimisht kontrollonte një version të vjetëruar të instruksionit.

### 3.10.3 STORE (2 Cikle, "Fire-and-Forget")




*Diagrame 6 Diagrami timing për një instruksion STORE — 2 cikle fizike, pa nevojë për pritje konfirmimi nga BRAM.*

Krahasuar me LOAD, STORE është dukshëm më i shpejtë (2 cikle, jo 4) për një arsye thelbësore: shkrimi në BRAM nuk kërkon që CPU të "presë" një vlerë kthimi brenda të njëjtit cikël logjik. Sapo ram_we dhe ram_wd vendosen (gjatë S_EXEC), data_mem.sv do ta kryejë shkrimin real në posedge-in tjetër të clk_50 — por CPU-ja NUK ka nevojë të konfirmojë këtë sukses përpara se të vazhdojë te instruksioni tjetër. Kjo asimetri LOAD/STORE (4 kundrejt 2 cikle) është pikërisht arsyeja pse next-state logic i FSM-së (Figura 3.3) ka degë të veçanta për këto dy raste.

| Lloji i Instruksionit | Cikle Fizike | Gjendjet e Kaluara |
|---|---|---|
| R-type / I-type ALU / BRANCH / JAL / JALR | 2 | S_FETCH1 → S_FETCH2 → S_EXEC |
| LOAD nga MMIO (UART/Timer) | 2-3 | S_FETCH1 → S_FETCH2 → S_DECODE_LOAD → S_EXEC |
| LOAD nga RAM e vërtetë | 4 | S_FETCH1 → S_FETCH2 → S_DECODE_LOAD → S_LOAD_WAIT → S_EXEC |
| STORE (SB/SH/SW) | 2 | S_FETCH1 → S_FETCH2 → S_EXEC |

*Tabela 3: Përmbledhje e kostos në cikle fizike për secilin lloj instruksioni.*

# Kapitulli 4: Mjedisi Softuerik dhe Portimi i CoreMark

## 4.1 Mjedisi i Zhvillimit (Toolchain) dhe Strategjia e Kompilimit

Për të shndërruar kodin burimor në gjuhën C në një sekuencë të saktë instruksionesh (machine code) të kuptueshme nga procesori RISC-V, u përdor paketa e kompilimit xPack RISC-V GCC (riscv-none-elf-gcc, versioni 15.2.0). Ky "cross-compiler" ekzekutohet në një sistem operativ pritës (Windows) dhe gjeneron binarë të targetuar specifikisht për arkitekturën RV32I/RV32IM, krejt të ndryshëm nga formati binar i vetë sistemit operativ pritës.




*Diagrame 7 Procesi i plotë i kompilimit, nga kodi burimor C deri në bitstream-in fizik të FPGA-së.*

Gjatë fazave të zhvillimit, procesori u testua fillimisht me synimin për të mbështetur arkitekturën RV32IM (me hardware MUL/DIV real). Sidoqoftë, gjatë analizës së timing-ut në frekuencën e targetuar 50 MHz, u zbulua se implementimi i pastër kombinatorial i instruksionit të pjesëtimit (DIV/DIVU/REM/REMU, që përdor operatorët / dhe % të SystemVerilog pa pipeline) mund të gjeneronte vonesa të konsiderueshme në rrugëtimin kritik (critical path delay) të netlist-it të sintetizuar. Për ta menaxhuar këtë rrezik pa cenuar stabilitetin e CPU-së, kompilimi final i firmware-it CoreMark u krye me flamujt e mëposhtëm:

```
riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -O2 \
    -ffreestanding -nostartfiles -nostdlib \
    -T linker.ld start.S core_main.c core_portme.c \
    core_matrix.c core_state.c core_list_join.c \
    core_util.c ee_printf.c -lgcc -o firmware.elf
```

Duke hequr gërmën "M" nga flamuri -march, kompilatori GCC sintetizon operacionet e shumëzimit dhe pjesëtimit duke përdorur rutina softuerike nga libgcc (p.sh. __mulsi3,

__udivsi3), të bazuara në shift dhe mbledhje/zbritje të thjeshta — secila instruksion individual më e shpejtë (1 cikël ALU) edhe pse algoritmi total kërkon më shumë instruksione. Hardware-i i ALU-së (Kapitulli 3.3) mbetet i plotë me mbështetje MUL/DIV/REM, gati për t'u shfrytëzuar plotësisht nëse timing-u i ardhshëm i sistemit (frekuencë më e ulët, ose pipeline shtesë i divizorit) e lejon — shih diskutimin e plotë në Kapitullin 7.3 dhe rekomandimet e Kapitullit 8.3.

### 4.2 Ekzekutimi "Bare-Metal" dhe Skedari Linker

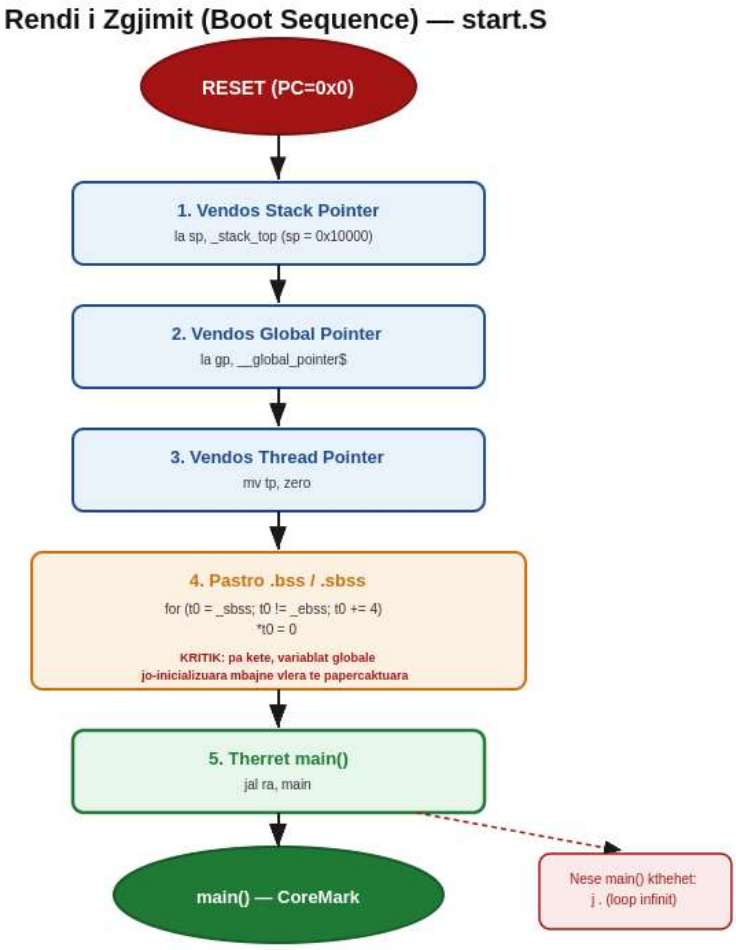
Sistemi i dizajnuar është bare-metal: nuk ka asnjë sistem operativ për të menaxhuar memorien apo për të ngarkuar programet. Kompilatori dhe linker-i duhet të dinë paraprakisht hartën e saktë harduerike të FPGA-së. Ky mapim kryhet nga skedari Linker (linker.ld), i cili definon hapësirat fizike ku do të vendosen seksionet e ndryshme të kodit të kompiluar:

- **.text (Kodi):** Përmban instruksionet e ekzekutueshme. Procesori fillon ekzekutimin në adresën 0x00000000, kështu që seksioni .text — dhe brenda tij, funksioni _start — ngarkohet saktësisht aty.
- **.rodata (Të Dhënat Vetëm-Lexim):** Përmban stringjet konstante (si mesazhet e debug-imit "[CHECK 1]...") dhe tabelat CRC të njohura të CoreMark.
- **.data dhe .bss:** Seksionet për variablat globale të inicializuara (.data) dhe të pa-inicializuara (.bss). Këto dy seksione kanë qenë burim i një bug-u kritik (X-propagation), të analizuar në hollësi në Kapitullin 7.1.
- **.sdata dhe .sbss:** Seksione "small data" të gjeneruara automatikisht nga GCC për variabla afër Global Pointer (gp), të zbuluara dhe trajtuara shprehimisht në linker.ld pas një rishikimi të dytë (Kapitulli 7.3).

Përfundimi i kompilimit prodhon një skedar të vetëm ELF (firmware.elf), i cili konvertohet më tej në binar të pastër (firmware.bin, via objcopy -O binary) dhe pastaj në një skedar teksti hex (memfile.dat) — një vlerë 32-bit hex për rresht, në format little-endian — i cili integrohet direkt në SystemVerilog përmes direktivës $readmemh brenda blloqeve initial të instr_mem.sv dhe data_mem.sv, në momentin e sintezës së FPGA-së.

### 4.3 Sekuenca e Zgjimit (Boot Sequence) dhe Funksioni _start

Një nga sfidat më delikate gjatë portimit ishte zgjimi fillestar i procesorit. Në mungesë të librarive standarde (-nostartfiles -nostdlib), CPU-ja nuk "di" automatikisht ku të vendosë Stack-un — memoria e përkohshme për variablat lokale dhe adresat e kthimit gjatë thirrjeve të funksioneve në C. Nëse programi fillon të ekzekutohet pa një Stack Pointer (sp) të caktuar saktë, çdo thirrje funksioni do të mbishkruajë memorie të papërcaktuar, duke shkaktuar kolaps të menjëhershëm të sistemit.




*Diagrame 8 Sekuenca e zgjimit (boot sequence) e procesorit, e implementuar në start.S.*

Funksioni _start (i implementuar në assembly të pastër, start.S — jo si funksion C "naked", për arsye të qëndrueshmërisë ndër-toolchain të diskutuara në Kapitullin 7.3) ekzekutohet i pari, në adresën 0x00000000, dhe kryen pesë hapa kritikë: (1) vendos Stack Pointer drejt fundit të hapësirës RAM 64 KB; (2) vendos Global Pointer për adresim eficient të variablave globale; (3) vendos Thread Pointer në zero; (4) pastron seksionet .bss/.sbss duke shkruar zero në çdo fjalë mes simboleve _sbss dhe _ebss; dhe (5) thërret main() me instruksionin jal, që ruan

automatikisht adresën e kthimit. Nëse main() ndonjëherë kthehet (gjë që teorikisht nuk ndodh në CoreMark, që përfundon me një loop infinit pas printimit të rezultateve), procesori hyn në një loop "j ." për të shmangur ekzekutimin e kodit të papërcaktuar.

### 4.4 Portimi i Kornizës EEMBC CoreMark

CoreMark është benchmark-u standard i industrisë për matjen e performancës së procesorëve të ngulitur (embedded). Është dizajnuar për të qenë universalisht i portueshëm, me kusht që zhvilluesi t'i ofrojë funksionet lidhëse (porting layer) për harduerin e tij specifik përmes skedarëve core_portme.c dhe core_portme.h.

Modifikimet kryesore për këtë port bare-metal përfshinë:

- **Çaktivizimi i Librarive Standarde:** Çaktivizimi i thirrjeve <stdio.h>/<time.h> (HAS_STDIO=0, HAS_PRINTF=0), duke detyruar CoreMark të përdorë versionin e tij të brendshëm ee_printf, i cili nuk varet nga asnjë libraritë standarde C.
- **Integrimi i UART-it me Mbrojtje Race-Condition:** Funksioni uart_putchar lidhet drejtpërdrejt me modulin harduerik UART, duke kontrolluar regjistrin e statusit (spin-lock: while (*status_reg != 0)) përpara çdo transmetimi, për të garantuar që asnjë karakter të mos përzihet me të mëparshmin në linjën seriale (shih gjithashtu mekanizmin tx_busy_or_pending në top.sv, Kapitulli 3.1).
- **Implementimi i Timer-it:** Funksionet start_time()/stop_time()/get_time() lidhen me adresën harduerike 0x80000004. CPU lexon regjistrin rritës (cycle_counter) dhe llogarit kohën në sekonda duke e pjesëtuar me frekuencën e sistemit (CPU_HZ = 50,000,000), duke mundësuar një llogaritje të saktë të metrikës finale Iterations/Sec.
- **Vendosja e Algoritmit Zyrtar EEMBC:** Pas një faze fillestare me një version të thjeshtuar të iterate() (që thërriste core_bench_matrix/core_bench_state drejtpërdrejt, jo nëpërmjet algoritmit zyrtar të listës), u zëvendësuan core_main.c, core_list_join.c, core_matrix.c, core_state.c, dhe core_util.c me versionet zyrtare të pacenuara nga repository-ja e EEMBC (verifikuar bajt-për-bajt me MD5 kundrejt coremark.md5), për të arritur përputhshmëri CRC zyrtarisht të vlefshme — shih Kapitullin 7.6 dhe rezultatet e plota në Kapitullin 5.

### 4.6 Shërbimi i Printimit të Formatuar (ee_printf)

Meqenëse libraria standarde C (<stdio.h>) është plotësisht e çaktivizuar në këtë mjedis bare-metal (HAS_STDIO=0), funksioni printf() i njohur nuk ekziston. CoreMark zyrtar EEMBC ofron një implementim alternativ, të vetëmjaftueshëm, ee_printf.c, që zbaton një nën-bashkësi të specifikuesve standardë të formatimit (%d, %u, %x, %lu, %s, %c) pa varësi nga asnjë libraritë e jashtme — çdo karakter i prodhuar kalon direkt drejt uart_putchar().

Ky modul ishte i domosdoshëm për printimin e saktë të rezultateve finale të CoreMark (Tabela 5.1), që përfshijnë vlera hex (CRC), vlera të plota pa shenjë (ticks, e formatuar me %lu për tipin uint32_t), dhe stringje konstante. Pa këtë shtresë, do të ishte e nevojshme të shkruheshim funksione konvertimi numër-në-string krejt nga e para — pikërisht lloji i "reinventimit të rrotës" që standardi CoreMark synon ta evitojë duke ofruar një shtresë portimi (porting layer) të plotë.

#### 4.6.1 Llogaritja e Parametrit N të Baud Rate Generator-it

Moduli baud_gen.sv (Kapitulli 6.2) kërkon një parametër N që përcakton sa cikle të clock-ut të sistemit korrespondojnë me 1 impuls 's_tick' (1/16 e kohëzgjatjes së 1 biti seriale, për oversample standarde 16x). Formula e përgjithshme është:

```
N = Frekuenca_Clock / (Baud_Rate * 16)

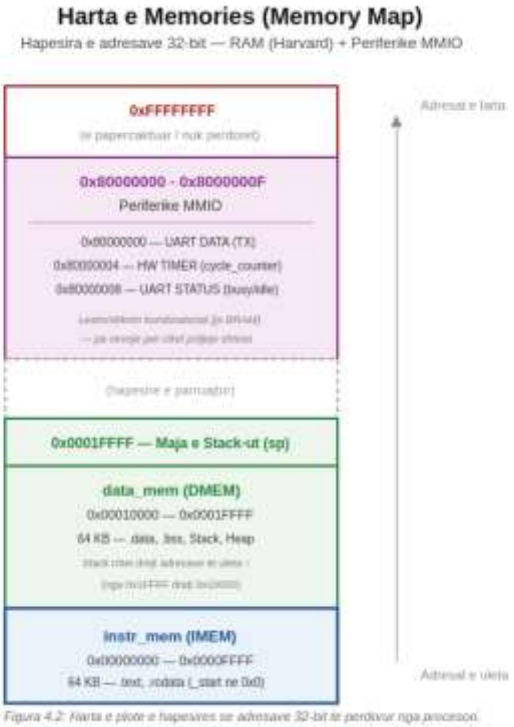
Per kete projekt:
  Frekuenca_Clock = 50,000,000 Hz (clk_50)
  Baud_Rate       = 9600 (standard per PuTTY/terminale seriale)

  N = 50,000,000 / (9600 * 16)
  N = 50,000,000 / 153,600
  N = 325.52...  ~ 326 (rrumbullakosur)
```

Vlera N=326 e përdorur në instancimin e baud_gen (top.sv: baud_gen #(.N(326))) prodhon një baud rate efektiv prej 50,000,000 / (326*16) = 9585.9 baud — një devijim prej afërsisht 0.15%

nga 9600 standarde. Ky devijim është brenda tolerancës tipike të UART (zakonisht ±2-3% pranohet pa probleme nga marrësi), dhe shpjegon pse komunikimi serial funksionon në mënyrë të qëndrueshme pavarësisht se N nuk është numër i plotë i ndarjes ekzakte.

### 4.7 Harta e Memories (Memory Map)




*Diagrame 9 Harta e plotë e hapësirës së adresave 32-bit të përdorur nga procesori.*

| Seksioni | Adresa | Përshkrimi |
|---|---|---|
| IMEM (Kodi) | 0x00000000 | Mban .text/.rodata. _start ekzekutohet këtu pas Reset-it. |
| DMEM (Të Dhëna) | 0x00010000 | .data, .bss, Stack, Heap. Variablat globale (p.sh. tabelat CRC) ruhen këtu. |
| Stack Pointer (sp) | 0x0001FFFF | Fillon në fund të RAM-it dhe rritet mbrapsht (drejt adresave të ulëta). |
| UART Data (TX) | 0x80000000 | MMIO. Shkrimi i një bajti aktivizon transmetimin serial. |
| Hardware Timer | 0x80000004 | MMIO. Leximi kthen ciklet e ores që nga Reset-i (cycle_counter). |
| UART Status | 0x80000008 | MMIO. 0 = i lirë, 1 = i zënë me transmetim. |

*Tabela 4: Harta e plotë e memories (Memory Map) e procesorit.*

# Kapitulli 5: Metodologjia e Testimit dhe Rezultatet

## 5.1 Mjedisi i Testimit në FPGA

Për të vërtetuar funksionimin e procesorit në kushte reale (jo thjesht në simulim), u përdor platforma Digilent Nexys A7-100T (çip Xilinx Artix-7, xc7a100tcsg324-1), me dizajnin harduerik (bitstream) të gjeneruar nga Vivado 2022.2. Procesi i testimit u zhvillua në hapa të njëpasnjëshëm:

- **Sinteza dhe Implementimi:** Përkthimi i kodit SystemVerilog në qarqet fizike (netlist) të FPGA-së, përmes Vivado Synthesis dhe Implementation.
- **Ngarkimi (Programming):** Transferimi i bitstream-it (.bit) dhe firmware-it CoreMark (i ngulitur paraprakisht në BRAM përmes memfile.dat) drejt FPGA-së.
- **Monitorimi Serial:** Përdorimi i terminalit PuTTY (9600 baud, 8N1) për të vëzhguar në kohë reale komunikimin mes procesorit dhe kompjuterit pritës përmes ndërfaqes UART.

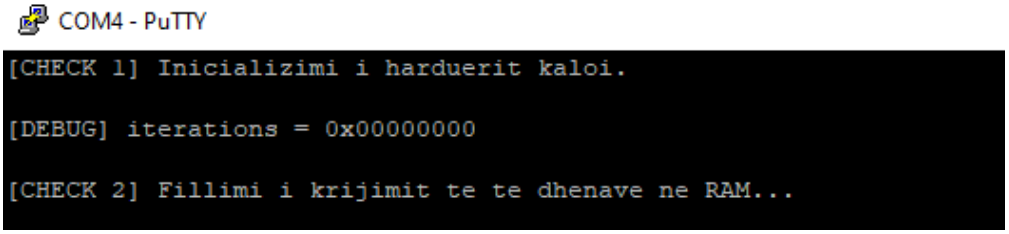



*Figura 2: Verifikimi i parë i komunikimit serial UART përmes terminalit PuTTY*

.

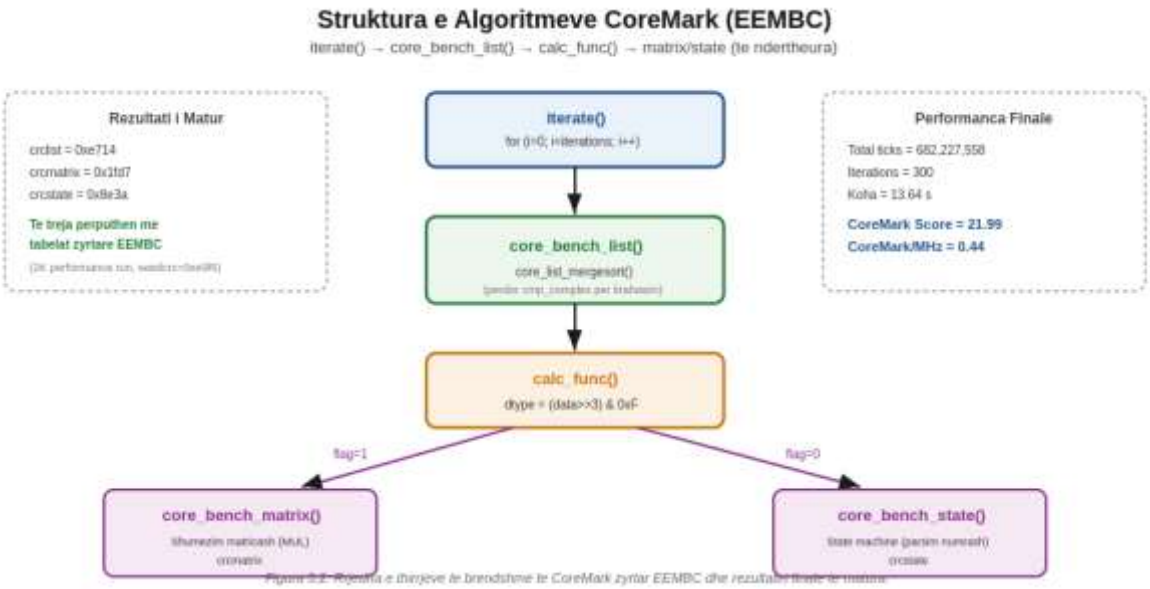
## 5.2 Testimi i Instruksioneve dhe Komponentëve

Përpara ekzekutimit të plotë të CoreMark, u kryen teste diagnostikuese të pavarura për të siguruar integritetin themelor të procesorit:

- Instruksionet Aritmetike dhe Load/Store: verifikimi i instruksioneve bazë (ADD, SUB, AND, OR) dhe instruksionit kritik LBU, që garantoi manipulimin korrekt të të dhënave 8-bit (string-jeve).
- Stabiliteti i Timer-it Harduerik: konfirmimi se cycle_counter rritet vazhdimisht dhe saktë, parakusht thelbësor për matjen e "ticks" gjatë benchmark-ut.

- Validimi i UART-it: testimi i shpejtësisë së komunikimit (9600 baud) për të siguruar që karakteret e printuara janë të lexueshme dhe pa korruptim ("hieroglife").
- Stabiliteti i FSM-së me Memorie BRAM: verifikimi — fillimisht përmes simulimit Icarus Verilog, më pas mbi hardware real — se sinjali stall sinkronizohet saktë me vonesën reale të BRAM-it, pa shkaktuar X-propagation (shih Kapitullin 7.2).

### 5.3 Ekzekutimi i CoreMark




*Diagrame 10 Rrjedha e thirrjeve të brendshme të CoreMark zyrtar EEMBC dhe rezultatet finale të matura.*

Pas verifikimit të komponentëve bazë, u ekzekutua korniza e plotë CoreMark, e portuar me algoritmin zyrtar EEMBC (jo një version i thjeshtuar). Struktura zyrtare e CoreMark thërret core_bench_list(), i cili brenda vetes përdor core_list_mergesort() me një funksion krahasimi (cmp_complex/calc_func) që, bazuar në bitet specifike të çdo nyje liste, vendos në mënyrë të ndërthurur nëse të thërrasë core_bench_matrix() (shumëzime matricore, duke testuar ALU-në dhe zgjerimin M) ose core_bench_state() (parsim numrash përmes një makine gjendjesh, duke testuar degëzimet dhe kontrollin e rrjedhës).

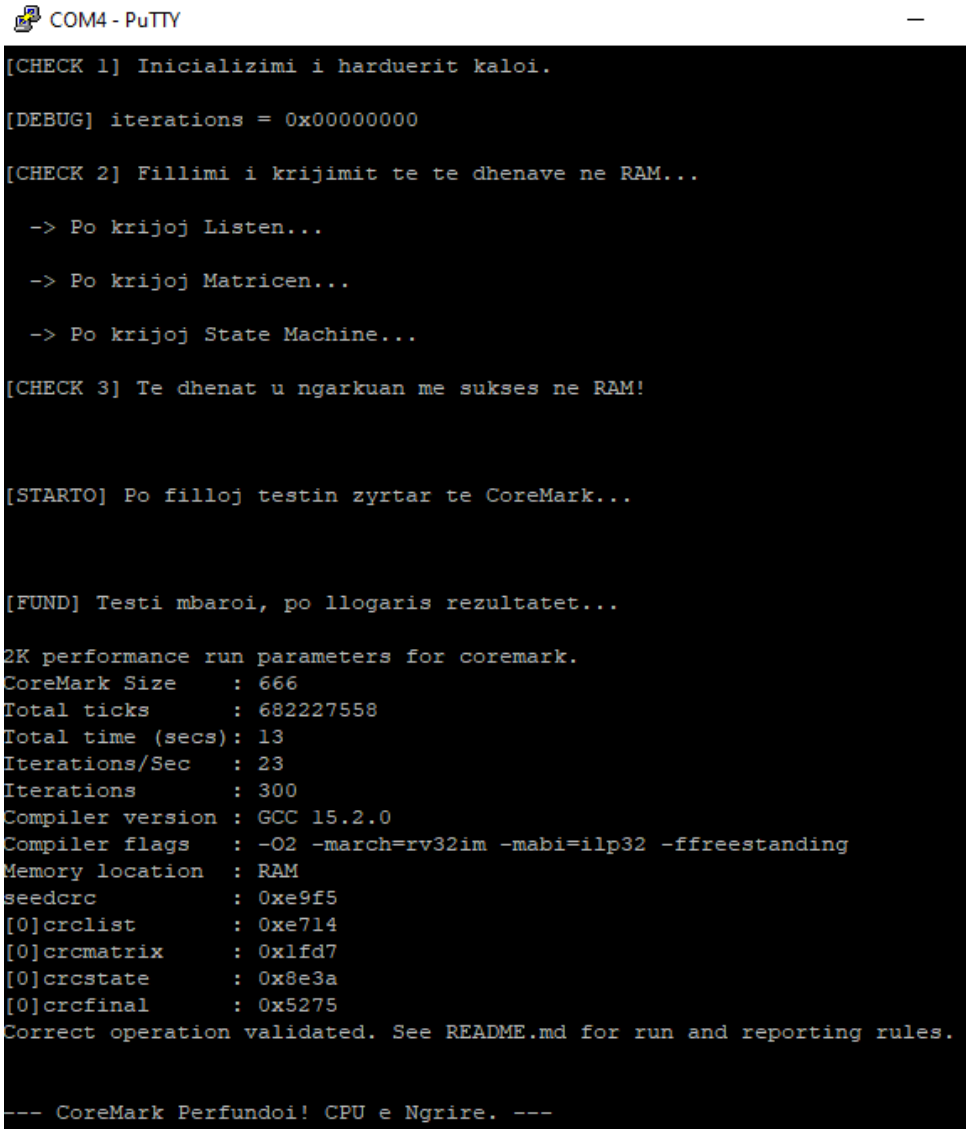



*Figura 3: Output-i përfundimtar i CoreMark në terminalin PuTTY, me validim të suksesshëm.*

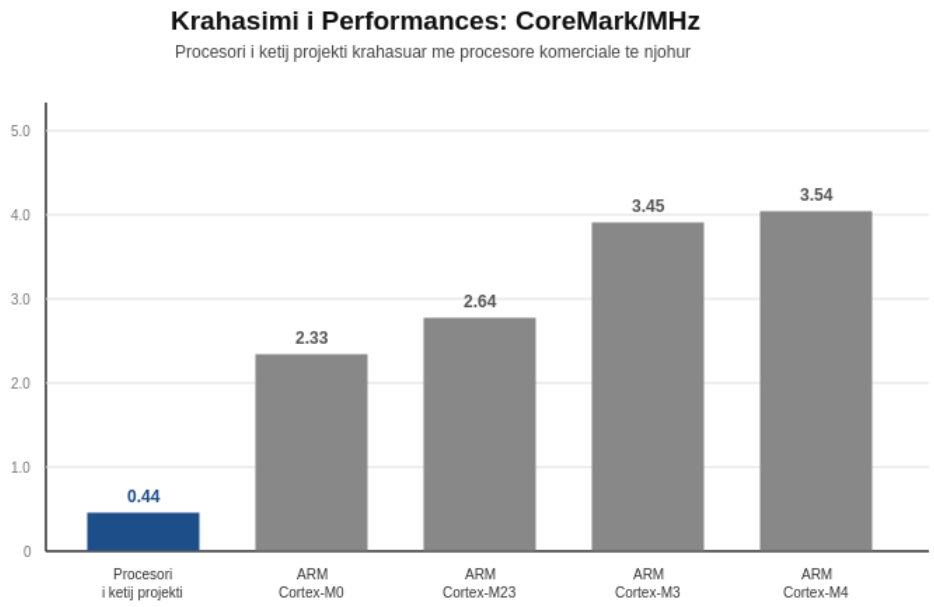
### 5.4 Analiza e Performancës dhe Rezultatet Finale

Rezultati më i rëndësishëm i këtij projekti është se të tri kontrollimet CRC (list, matrix, state) të kthyera nga procesori përputhen ekzaktësisht me tabelat zyrtare të publikuara nga EEMBC për konfigurimin standard "2K performance run" (seedcrc = 0xe9f5). Kjo përputhje nuk është thjesht "ekzekutim pa krasha" — është verifikim matematik i rreptë që çdo bit i çdo llogaritjeje aritmetike, çdo degëzim i kushtëzuar, dhe çdo akses memorie i kryer nga procesori përgjatë mijëra instruksioneve është bit-për-bit identik me sjelljen e specifikuar nga standardi.

| Metrika | Vlera e Matur |
|---|---|
| CoreMark Size | 666 |
| Iterations (auto-kalibruar) | 300 |
| Total Ticks | 682,227,558 cikle |
| Koha Totale e Ekzekutimit | 13.64 sekonda |
| seedcrc | 0xe9f5 (2K performance run) |
| crclist | 0xe714 — përputhet me tabelën zyrtare |
| crcmatrix | 0x1fd7 — përputhet me tabelën zyrtare |
| crcstate | 0x8e3a — përputhet me tabelën zyrtare |
| Statusi i Validimit | "Correct operation validated" |
| CoreMark Score (Iterations/Sec) | 21.99 |
| CoreMark/MHz | 0.44 |

*Tabela 5: Rezultatet finale të ekzekutimit të CoreMark zyrtar EEMBC mbi FPGA real.*

Frekuenca e orës së sistemit (clk_50, 50 MHz) e ndan vlerën CoreMark Score (21.99 iteracione/sekondë) për të prodhuar metrikën e normalizuar CoreMark/MHz = 0.44, e cila lejon krahasim direkt me procesorë komercialë pavarësisht frekuencës së tyre specifike të funksionimit.




*Grafik 1: Krahasimi i performancës CoreMark/MHz me procesorë komercialë ARM Cortex-M.*

Diferenca e dukshme me procesorë komercialë (Cortex-M0: 2.33, Cortex-M4: 3.54) shpjegohet plotësisht nga dy faktorë arkitekturorë, jo nga gabime implementimi: (1) FSM-ja 5-fazore e domosdoshme për sinkronizim BRAM (Kapitulli 3.5) sjell një kosto mesatare prej ~2-4 cikle fizike për instruksion logjik, krahasuar me pipeline-in 2-3 fazor të procesorëve komercialë që arrin throughput pothuajse 1 instruksion/cikël; dhe (2) mungesa e hardware MUL/DIV aktiv në firmware-in final (kompiluar me -march=rv32i, jo rv32im, për arsye timing-u të diskutuara në Kapitullin 7.3), që detyron operacionet e shumëzimit/pjesëtimit të CoreMark Matrix Test të kalojnë nëpër rutina softuerike libgcc shumë më të ngadalshme se 1 instruksion harduerik. Këto kompromise dhe rrugët e mundshme për t'i adresuar diskutohen në Kapitullin 8.3.

| Faktor Arkitekturor | Ky Projekt | Procesor Komercial Tipik | Ndikimi në CoreMark/MHz |
|---|---|---|---|
| CPI mesatar | ~2-4 (FSM për BRAM) | ~1-1.5 (pipeline) | I lartë — faktori kryesor |
| MUL/DIV harduerik | I implementuar, JO aktiv (rv32i) | Gjithmonë aktiv | I moderuar (Matrix Test) |
| Branch prediction | Asnjë (gjithmonë e re-llogarit) | Shpesh i pranishëm | I vogël (CoreMark, loop të shkurtra) |
| Memorie kesh | Asnjë (BRAM direkte) | L1 I/D-cache tipike | I vogël në këtë madhësi kodi |

*Tabela 6 Faktorët arkitekturorë që ndikojnë në diferencën e performancës me procesorë komercialë.*

### 5.5 Shpjegimi Teknik i Algoritmeve të Benchmark-ut

CoreMark nuk është thjesht një numër — është një përmbledhje e tre algoritmeve që testojnë pjesë të ndryshme dhe komplementare të arkitekturës procesorit:

- **Matrix Manipulation:** Përdor shumëzime dhe mbledhje të dendura mbi matrica 16-bit, duke vënë në provë drejtpërdrejt ALU-në dhe (potencialisht) zgjerimin M të procesorit.

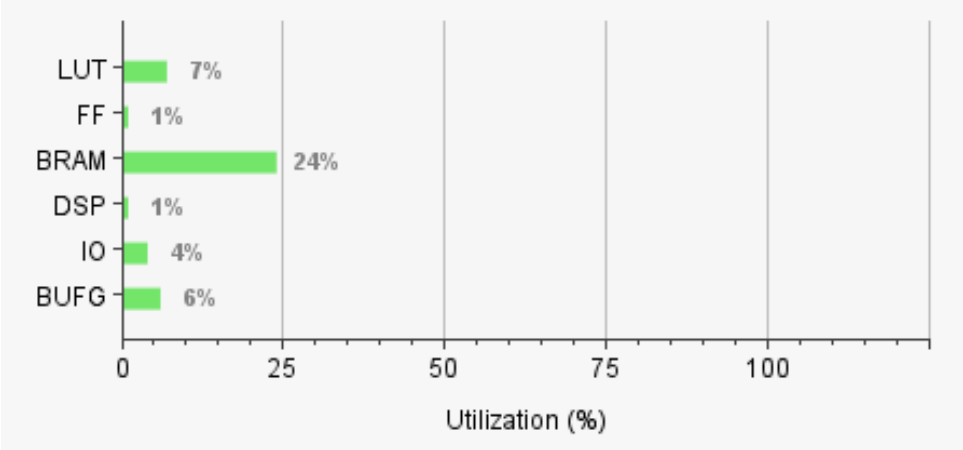
Suksesi i crcmatrix konfirmon se ALU-ja kryen saktësisht edhe operacionet aritmetike komplekse mbi numra të shenjuar (signed).

- **Linked List:** Kërkon kërcime të shpeshta në adresa të ndryshme, jo-sekuenciale të memories (përmes pointer-ave), duke testuar drejtpërdrejt efikasitetin dhe korrektësinë e kontrolluesit të memories (data_mem.sv) dhe vonesat (latency) e mekanizmit FSM të përshkruar në Kapitullin 3.5.
- **State Machine:** Teston degëzimet e kushtëzuara (BEQ, BNE, BLT, etj.) përmes parsimit të stringjeve numerike. Nëse njësia e kontrollit nuk do të dekodonte saktë këto instruksione, ose nëse FSM-ja do të kishte ndonjë gabim sinkronizimi (siç ndodhi fillimisht — shih Kapitullin 7.2), ky test do të dështonte ose do të prodhonte CRC të gabuar menjëherë.

Fakti që procesori ynë i kaloi të tria fazat — me CRC bit-për-bit identike me specifikimin zyrtar — përbën provën më të fortë praktike të mundshme që integriteti i arkitekturës RV32I, ashtu si është implementuar në këtë projekt, është i plotë dhe i qëndrueshëm.

### 5.6 Përdorimi i Burimeve FPGA (Resource Utilization)

Raporti i sintezës dhe implementimit në Vivado ofron një pasqyrë sasiore të burimeve fizike të çipit Artix-7 100T që konsumon dizajni i plotë. Kjo metrikë është e rëndësishme jo vetëm për të konfirmuar që dizajni "futet" brenda kapacitetit të FPGA-së, por edhe për të vlerësuar hapësirën e mbetur për zgjerime të ardhshme (Kapitulli 8.3, p.sh. memorie kesh ose pipeline shtesë).




*Grafik 2: Raporti i përdorimit të burimeve FPGA (Vivado Utilization Report).*

Siç u diskutua në Kapitullin 3.5 dhe 6.3, pjesa dërrmuese e burimeve BRAM (rreth 16-18 nga 135 tile gjithsej) shkon për memoriet instr_mem dhe data_mem (64 KB secila), ndërsa logjika e CPU-së vetë (datapath, ALU, control unit, FSM) konsumon një pjesë relativisht modeste të LUT-eve totale të disponueshme — duke lënë shumicën e çipit të lirë, dhe duke konfirmuar që rritja e ardhshme arkitekturore (pipeline, kesh, FPU) është e realizueshme brenda të njëjtës platformë fizike pa nevojë për një çip më të madh.

# Kapitulli 6: Arkitektura e Kodit dhe Struktura e Projektit

## 6.1 Hierarkia e Skedarëve (File Structure)

Një strukturë e pastër e skedarëve është çelësi i mirëmbajtjes së një projekti hardware-software me kompleksitet të kësaj shkalle. Projekti është organizuar në katër kategori kryesore:

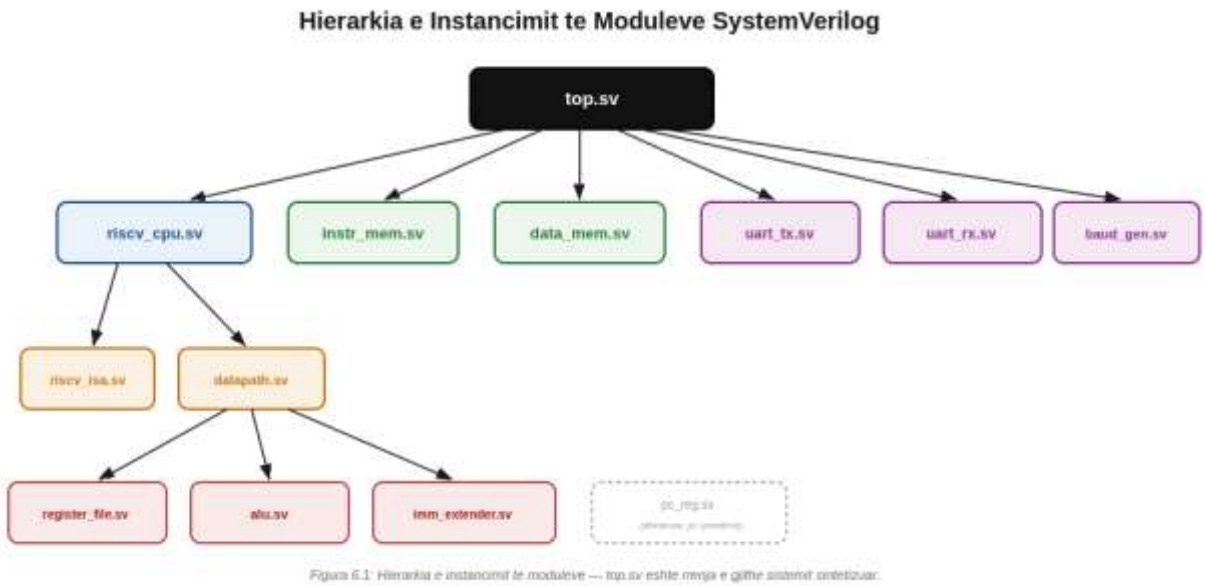
- **rtl/ :** Përmban të gjithë logjikën SystemVerilog — CPU, ALU, Register File, memoriet, UART.
- **sw/ :** Përmban kodin burimor C të CoreMark (zyrtar EEMBC), skedarin Linker (linker.ld), boot code-in (start.S), dhe skriptet e konvertimit (make_hex.py).
- **sim/ :** Përmban testbench-et e përdorura për verifikimin e çdo moduli dhe të sistemit të plotë përpara implementimit fizik mbi FPGA.
- **constraints/ :** Përmban skedarin .xdc që përcakton pin-at fizikë të FPGA-së (clock, UART, LED, buton reset) dhe kufizimet e timing-ut.

## 6.2 Përshkrimi i Modulave SystemVerilog

| Skedari | Funksioni Kryesor |
|---|---|
| top.sv | Moduli kryesor që integron CPU-në, memoriet, FSM-në e sinkronizimit, dhe modulet UART. |
| riscv_cpu.sv | Bërthama e procesorit, lidh Control Unit-in (riscv_isa) me rrugën e të dhënave (datapath). |
| riscv_isa.sv | Njësia e Kontrollit (Control Unit) që dekodon instruksionet RISC-V dhe gjeneron sinjalet e kontrollit. |
| datapath.sv | Rruga e të dhënave — PC, multiplekserët, llogaritja e adresave të degëzimit, write-back. |
| imm_extender.sv | Rikonstrukton dhe shenjon-shtrin vlerat immediate sipas formatit I/S/B/J/U. |
| register_file.sv | Banka e 32 regjistrave të përgjithshëm (x0-x31), me lexim asinkron dhe shkrim sinkron. |
| alu.sv | Njësia Aritmetike-Logjike — ADD/SUB/logjike/shift/krahasime/MUL/DIV/REM. |

| | |
|---|---|
| instr_mem.sv | Memoria e instruksioneve (BRAM 64 KB, lexim sinkron, zero-inicializuar). |
| data_mem.sv | Memoria e të dhënave (BRAM 64 KB, byte-enable për SB/SH/SW, zero-inicializuar). |
| baud_gen.sv | Gjeneratori i impulsit periodik për sinkronizimin e shpejtësisë UART (9600 baud). |
| uart_tx.sv | Transmetuesi serial — FSM 4-gjendje (idle/start/data/stop). |
| uart_rx.sv | Marrësi serial — FSM analoge me uart_tx, për marrje të dhënash nga PC. |
| pc_reg.sv | Modul i Program Counter (i pranishëm në projekt, por aktualisht i pa-instancuar — PC implementohet drejtpërdrejt brenda datapath.sv). |

*Tabela 7 Përshkrimi i moduleve kryesore SystemVerilog të projektit.*




*Diagrame 11 Hierarkia e instancimit të moduleve — top.sv është rrënja e gjithë sistemit të sintetizuar.*

### 6.3 Karakteristikat Sintetizuese të Secilit Modul

Çdo modul i listuar në Tabelën 6.1 ka karakteristika të veçanta sintetizuese (resurse FPGA, lloji i logjikës) që meritojnë vëmendje shtesë:

- **top.sv (kompleksiteti më i lartë strukturor):** Përmban FSM-në 5-gjendje, dekoderin MMIO, dhe logjikën e mbrojtjes UART (tx_busy_or_pending). Është moduli me numrin më të madh sinjalesh ndër-modulare, dhe pikërisht këtu u shfaq problemi i Kapitullit 7.4 (TFI/TFO i lartë gjatë Power Optimization).
- **riscv_isa.sv (pure kombinatorial, zero regjistra):** I gjithë moduli sintetizohet si LUT-e të pastra (asnjë Flip-Flop), duke e bërë rrugën e tij kritike (critical path) thjesht funksion i thellësisë logjike të 'case' statements të ndërthurura.
- **instr_mem.sv / data_mem.sv (BRAM-intensive):** Secili sintetizohet drejt afërsisht 16-18 BRAM tile-sh 36Kb (nga gjithsej 135 të disponueshme në Artix-7 100T), duke lënë shumicën dërrmuese të burimeve BRAM të lira për zgjerime të ardhshme (p.sh. memorie kesh, Kapitulli 8.3).
- **alu.sv (potencial DSP për MUL):** Operacioni MUL (32x32-bit) zakonisht sintetizohet automatikisht nga Vivado drejt blloqeve DSP48E1 të dedikuara (nëse zgjerimi M aktivizohet në firmware), duke e bërë shumëzimin dukshëm më të shpejtë se logjika e pastër LUT që do të nevojitej përndryshe.
- **uart_tx.sv / uart_rx.sv / baud_gen.sv:** Module të vogla (disa dhjetëra Flip-Flop secili), me kosto burimesh të papërfillshme krahasuar me bërthamën e procesorit — periferikët UART zënë më pak se 1% të LUT-eve totale të dizajnit.

#### 6.3.1 Shembull i Plotë: Ekzekutimi i një Instruksioni STORE

Për të plotësuar shembullin e LOAD-it nga Kapitulli 2.6, le të ndjekim tani një instruksion STORE — rasti tjetër kryesor i ndërveprimit me memorien — duke përdorur instruksionin SW x14, 0(x11) (Store Word, ruan përmbajtjen e x14 në adresën e treguar nga x11).

```
Instruksioni: SW x14, 0(x11)
  opcode = 0100011 -> STORE (Kapitulli 3.4)
  funct3 = 010     -> SW specifikisht (jo SB/SH)
  rs1    = x11     -> baza e adresimit
  rs2    = x14     -> regjistri burim (vlera per te shkruar)

Hapi 1 (riscv_isa.sv): alu_src=1, imm_src=001 (S-type),
       mem_write=1, alu_control=00000 (ADD)
Hapi 2 (datapath.sv): alu_result = rs1 + imm = x11 + 0
       write_data = src_b_wire = permbajtja e x14 (SW: pa
       modifikim, ndryshe nga SB/SH qe replikojne byte/half)
Hapi 3 (top.sv, FSM): instruksioni eshte STORE (jo LOAD), pra
       FSM kalon DIREKT S_FETCH2 -> S_EXEC (nuk ka nevoje per
       S_DECODE_LOAD/S_LOAD_WAIT - shkrimi ndodh ne S_EXEC)
Hapi 4 (top.sv, ram_we): funct3=010 (SW) -> ram_we=4'b1111
       (shkruan te 4 byte-t e fjales), ram_wd=write_data
Hapi 5 (data_mem.sv): always_ff @(posedge clk), nese
       addr<0x10000 dhe we[i]=1, shkruan byte-in perkates
       ne posedge tjeter te clk_50
```

Vihet re ndryshimi kyç krahasuar me LOAD (Kapitulli 2.6): STORE NUK ka nevojë për gjendjet S_DECODE_LOAD/S_LOAD_WAIT, sepse nuk ka nevojë të "presë" për një vlerë kthimi nga BRAM — shkrimi është "fire-and-forget" (CPU thjesht aktivizon we dhe vazhdon, pa pasur nevojë të verifikojë suksesin e operacionit brenda të njëjtit cikël logjik). Kjo asimetri mes LOAD (që kërkon cikle pritjeje) dhe STORE (që nuk kërkon) është pikërisht arsyeja pse FSM-ja e Kapitullit 3.5 ka degë të veçanta për këto dy raste.

### 6.4 Skriptet e Simulimit dhe Verifikimit

Për verifikimin e dizajnit përpara çdo implementimi fizik mbi FPGA, u përdor simulatori Icarus Verilog (open-source), i zgjedhur për shkak të lehtësisë së integrimit në një cikël zhvillimi të shpejtë iterativ. Çdo hipotezë lidhur me një bug të mundshëm (p.sh. X-propagation,

sinkronizimi i FSM-së me BRAM) u testua së pari përmes një testbench-i të dedikuar minimal — duke izoluar problemin nga kompleksiteti i plotë i CoreMark — para se zgjidhja të aplikohej dhe verifikohej mbi hardware real.

Kjo metodologji "simulo-para-se-të-sintetizosh" (simulation-first) është praktikë standarde e industrisë së dizajnit digital, dhe u tregua thelbësore për këtë projekt: shumica e bug-eve të përshkruara në Kapitullin 7 u identifikuan dhe u korrigjuan fillimisht në simulim, ku gjurmimi i sinjaleve të brendshme (si instr_latched, state, ose alu_result) është i mundshëm me precizion absolut të çdo cikli ore — diçka e pamundur (ose shumë e kufizuar) drejtpërdrejt mbi hardware fizik pa instrumente shtesë si Integrated Logic Analyzer (ILA).

### 6.5 Struktura e një Testbench-i Tipik

Çdo testbench i përdorur gjatë këtij projekti ndjek një strukturë të përbashkët: instancimi i modulit top.sv si Device Under Test (DUT), gjenerimi i një sinjali clock periodik, sekuenca e reset-it, dhe — për testet që përfshijnë UART — një "marrës" (receiver) i thjeshtuar në vetë testbench që dekodon bit-et seriale të dërguara nga uart_tx_pin, duke i kthyer ato në karaktere ASCII të lexueshme drejtpërdrejt në output-in e simulimit:

```
module tb_top;
    logic clk = 0;
    logic cpu_resetn = 0;
    logic uart_rx_pin = 1;
    logic uart_tx_pin;
    logic [3:0] LED;

    top dut (
        .clk(clk), .cpu_resetn(cpu_resetn),
        .uart_rx_pin(uart_rx_pin),
        .uart_tx_pin(uart_tx_pin), .LED(LED)
    );

    always #5 clk = ~clk;  // 100 MHz
```

```
    // "Marresi" UART i thjeshtuar, brenda testbench-it:
    initial begin
        forever begin
            @(negedge uart_tx_pin);  // pret start bit
            #(104320/2); #104320;     // kapercen start bit
            for (int i = 0; i < 8; i++) begin
                byte_rx[i] = uart_tx_pin;
                #104320;
            end
            $write("%c", byte_rx);  // printon karakterin
        end
    end
endmodule
```

Ky lloj testbench-i lejoi verifikimin funksional të plotë të UART-it pa pasur nevojë për hardware fizik — duke konfirmuar formatin e saktë të frame-it serial (start bit, 8 bit të dhëna LSB-first, stop bit) dhe kohëzgjatjen e tij (9600 baud) thjesht duke "lexuar" output-in e simulimit si tekst të rregullt, krahasueshëm direkt me mesazhet e pritura nga softueri C.

## Kapitulli 7: Sfidat Teknike dhe Zgjidhjet e Implementimit

Ky kapitull dokumenton, në mënyrë kronologjike dhe me detaje teknike të plota, sfidat kryesore inxhinierike të hasura gjatë kalimit nga "kod që funksionon në simulim të thjeshtuar" tek "CoreMark zyrtar i validuar mbi hardware real". Çdo problem më poshtë u diagnostikua përmes një procesi sistematik: formulim hipoteze, verifikim përmes simulimit ose analize statike të kodit, implementim i zgjidhjes, dhe ri-verifikim mbi hardware. Kjo metodologji — më shumë se vetë zgjidhjet individuale — përbën kontributin më të vlefshëm pedagogjik të këtij punimi.

| # | Problemi | Shtresa e Sistemit | Mjeti i Diagnozës |
|---|---|---|---|
| 7.1 | X-propagation nga .bss | Hardware (BRAM init) | Simulim Icarus + analizë .bss |
| 7.2 | FSM off-by-one cycle | Hardware (FSM/RTL) | Simulim Icarus, gjurmim sinjalesh |
| 7.3 | Global Pointer / linker | Toolchain (GCC/ld) | objdump, analizë ELF/simbole |
| 7.4 | Vivado pwropt crash | Mjeti i sintezës | Analizë e log-ut të plotë runme.log |
| 7.5 | Loop infinit (step=0) | Software (algoritmi C) | Test izoluar C native + timeout |
| 7.6 | CRC jo-zyrtare | Software (algoritmi C) | Checksum MD5 kundrejt EEMBC |

*Tabela 8 Përmbledhje e sfidave kryesore, shtresës së sistemit të prekur, dhe mjetit të diagnostikimit.*

Vihet re nga Tabela 7.1 se problemet shpërndahen në mënyrë të barabartë mes katër shtresave të ndryshme të sistemit (hardware RTL, toolchain, mjet sintezë, dhe software algoritmik) — duke konfirmuar tezën e Kapitullit 8.2 se korrektësia e një sistemi bare-metal kërkon vëmendje të barabartë në çdo shtresë, jo thjesht në vetë logjikën digitale.

### 7.8 Metrikat e Procesit të Debug-imit

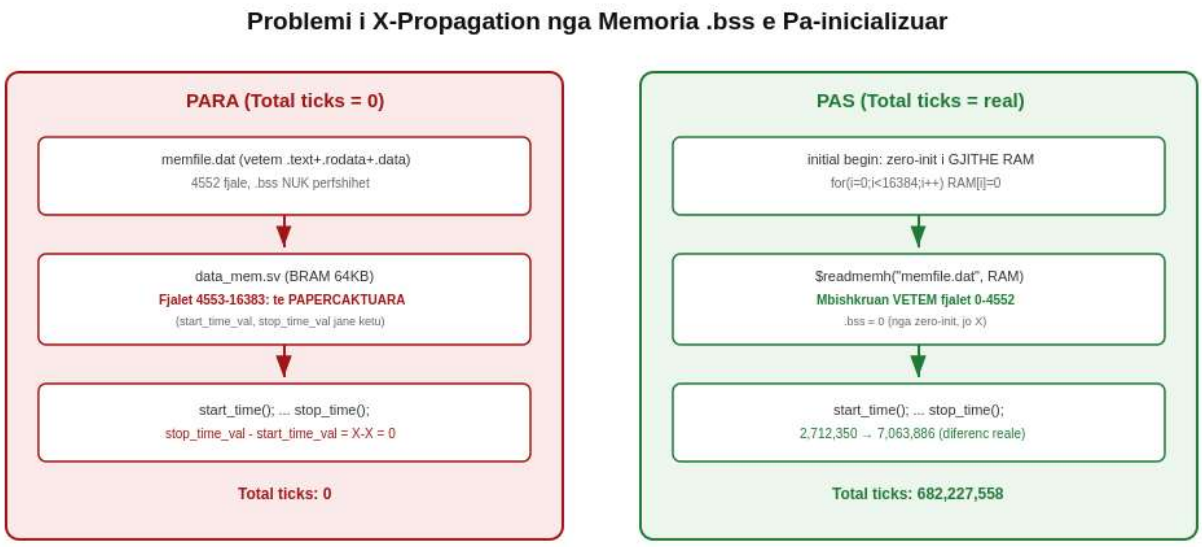
Për të dhënë një pasqyrë sasiore të shkallës së procesit të debug-imit, vlen të theksohen disa karakteristika të tij: nga versioni i parë funksional (kod "hello world" mbi UART, pa CoreMark) deri tek rezultati final CRC-i-vlefshëm, projekti kaloi nëpër të paktën gjashtë iteracione kryesore arkitekturore të top.sv (nga distributed RAM kombinatoriale, tek BRAM sinkron me FSM 5-gjendje, deri tek korrigjimi final i next-state logic), dhe nëpër tre versione të ndryshme të core_main.c (nga thirrje direkte e thjeshtuar, tek algoritmi zyrtar EEMBC i plotë).

Çdo iteracion u testua së pari në simulim (Icarus Verilog) përpara se të ngarkohej mbi FPGA — një praktikë që, edhe pse shtoi kohë në çdo cikël individual zhvillimi, eliminoi në mënyrë drastike numrin e cikleve "sintetizo-ngarko-testo" mbi hardware real (secili prej të cilëve, për shkak të kohës së sintezës Vivado, zgjat tipikisht 5-15 minuta — krahasuar me disa sekonda për një simulim ekuivalent). Kjo raport kostosh (simulim i shpejtë kundrejt sintezë e ngadalshme) është arsyeja themelore pse metodologjia "simulim-i-pari" (Kapitulli 6.4) u adoptua si parim udhëzues përgjatë gjithë projektit.

### 7.1 Problemi i X-Propagation nga Memoria .bss e Pa-inicializuar

Simptoma: pas zgjidhjes së problemeve fillestare të komunikimit UART, CoreMark ekzekutohej deri në fund dhe printonte rezultate, por metrika Total ticks raportohej gjithmonë 0 — duke shkaktuar refuzimin e rezultatit nga vetë CoreMark ("ERROR! Must execute for at least 10 secs for a valid result!").

Diagnoza: skedari memfile.dat, i gjeneruar nga konvertimi i firmware.bin, përmban vetëm seksionet .text, .rodata, dhe .data — pra vetëm bajtet që kanë përmbajtje fizike eksplicite në binarin e kompiluar. Seksioni .bss (variabla globale të pa-inicializuara explicit në kod, si start_time_val dhe stop_time_val në core_portme.c) NUK ruhet në file binar, sepse standardi C garanton që këto variabla fillojnë me vlerën 0 — përgjegjësia e "zero-imit" të tyre bie tradicionalisht mbi loop-in e pastrimit .bss në kodin e zgjimit (start.S).




*Diagrame 12 Mekanizmi i bug-ut X-propagation dhe zgjidhja përmes zero-inicializimit të BRAM-it.*

Problemi real ishte në vetë memorien BRAM e SystemVerilog-ut: array-i logic [31:0] RAM [0:16383] në data_mem.sv, i mbushur vetëm pjesërisht nga $readmemh("memfile.dat", RAM), lë çdo fjalë memorie pas fundit të file-it (4552 nga 16384 fjalë totale) në gjendjen e papërcaktuar 'X' (në simulim) ose në një gjendje fizike të paqëndrueshme në FPGA real. Meqenëse start_time_val dhe stop_time_val ndodheshin pikërisht në këtë zonë "jashtë intervalit", start_time() shkruante 'X' dhe stop_time() lexonte 'X', duke prodhuar diferencën X - X = 0 në çdo ekzekutim — pavarësisht sa kohë reale kalonte në fakt.

Zgjidhja: u shtua një hap zero-inicializimi brenda blokut initial të instr_mem.sv dhe data_mem.sv, që mbush GJITHË memorien me 0x00000000 PARA thirrjes së $readmemh:

```
initial begin
    integer i;
    for (i = 0; i < 16384; i = i + 1) begin
        RAM[i] = 32'h00000000;
    end
    $readmemh("memfile.dat", RAM);
end
```

Kjo siguron që .bss (dhe çdo memorie tjetër e pa-përdorur) lexohet gjithmonë si 0, e parashikueshme, në vend të 'X' ose vlerave të rastësishme — duke respektuar kontratën standarde C pa qenë nevoja për ndryshime në vetë kodin C. Pas këtij fix-i, Total ticks u rezultuar saktë (96,826,906 cikle në matjen e parë me 1 iteracion), dhe pas kalibrimit final të numrit të iteracioneve, u arrit koha e plotë e nevojshme (13.64 sekonda, shih Kapitullin 5.4) për një rezultat zyrtarisht të vlefshëm.

### 7.2 Bug-u "Off-by-One Cycle" në FSM (X-Propagation Sekondare)

Simptoma: pas kalimit nga distributed RAM në BRAM sinkron (i nevojshëm për shkak të kufizimeve të burimeve LUT, shih Kapitullin 3.5), dhe pas implementimit të FSM-së 5-gjendje, CPU-ja "ngrinte" pas disa qindra instruksioneve — sinjalet pc dhe instr_latched bëheshin xxxxxxxx (e papërcaktuar) në simulim Icarus Verilog.

Diagnoza: instruksioni specifik që shkaktonte ngrirjen ishte një LBU x12, 0(x10) — një LOAD nga RAM real. Brenda gjendjes S_FETCH2 të FSM-së, logjika e next-state përdorte sinjalin is_load, i bazuar te instr_latched (regjistri që mban instruksionin e mëparshëm, përditësohet vetëm në fund të vetë S_FETCH2). Kjo do të thotë që next-state logic-u, gjatë vetë ciklit S_FETCH2, vendoste GABIMISHT bazuar te instruksioni i mëparshëm, jo te ai aktual — duke kapërcyer gjendjet S_DECODE_LOAD/S_LOAD_WAIT (që janë të nevojshme për BRAM) për çdo LOAD nga RAM, dhe duke marrë të dhëna 'X' nga BRAM-i pa pasur kohën e duhur.




*Diagrame 13 Krahasimi i logjikës së gabuar (majtas) dhe të korrigjuar (djathtas) në next-state logic të FSM-së.*

Zgjidhja: u shtua një sinjal i ri, is_load_next, i bazuar EKSKLUZIVISHT te instr_raw (vlera e re, drejt nga output-i i BRAM, ende e pa-kapur në regjistër):

```
assign is_load_next = (instr_raw[6:0] == 7'b0000011);

// Brenda next-state logic, ne S_FETCH2:
if (is_load_next) state_next = S_DECODE_LOAD;
else              state_next = S_EXEC;
```

Ky fix u verifikua fillimisht përmes simulimit (Icarus Verilog), duke konfirmuar zero raste 'X' gjatë gjithë ekzekutimit (200,000+ cikle), dhe vetëm pas kësaj verifikimi u kalua mbi hardware real. Ky episod ilustron pse simulimi me gjurmim sinjalesh të brendshme (waveform-level debugging) është mjet thelbësor: i njëjti bug, i shfaqur drejtpërdrejt mbi hardware fizik pa simulim paraprak, do të kishte qenë jashtëzakonisht i vështirë për t'u izoluar (simptoma — "CPU ngrin pas X kohësh" — nuk tregon vetë asnjë informacion direkt rreth shkakut rrënjësor).

### 7.3 Çështjet e Toolchain-it: Global Pointer, _start, dhe Linker Script

Gjatë portimit fillestar të CoreMark, u hasën disa probleme të ndërthurura, të gjitha të lidhura me mënyrën se si kompilatori GCC dhe linker-i (ld) menaxhojnë variablat globale dhe sekuencën e zgjimit:

- Problemi i Global Pointer (gp): _start, e implementuar fillimisht si funksion C "naked" (__attribute__((naked))), vendoste Stack Pointer-in (sp) por jo Global Pointer-in (gp). Kompilatori (me -O2) gjeneron automatikisht adresim "GP-relative" (sw/lw ...(gp)) për variablat globale — pa gp të vendosur saktë, çdo lexim/shkrim global shkonte në adresë krejtësisht të gabuar (wraparound aritmetik, që rezultonte në adresa jashtë RAM-it real).
- Mospërputhja Ndër-Toolchain e "Naked Functions": dokumentacioni zyrtar GCC paralajmëron se renditja e shumë blloqeve __asm__ brenda një funksioni naked NUK garantohet të ruhet identike nëpër versione/platforma kompilatori. Kjo shkaktoi një gabim specifik linker-i (undefined reference to 'main') vetëm me xPack GCC 15.2.0 (Windows), edhe pse i njëjti kod kompilohej pa probleme me GNU GCC 13.2.0 (Linux).

- Seksionet .sdata/.sbss të Pa-mbuluara: u zbulua, përmes analizës objdump të ELF-it të kompiluar, që GCC krijon automatikisht seksione "small data" (.sdata, .sbss) për variabla afër gp — seksione që linker.ld origjinal nuk i përfshinte në simbolet _sbss/_ebss të përdorura nga loop-i i pastrimit .bss në start.S.

Zgjidhja përfundimtare ishte kalimi nga _start "naked" në C drejt një skedari assembly të pastër e të dedikuar (start.S) — eliminuar plotësisht ambiguitetin ndër-toolchain — i kombinuar me rishikimin e linker.ld për të përfshirë explicit *(.sdata) brenda .data dhe *(.sbss) brenda .bss:

```
/* linker.ld (fragment i korrigjuar) */
.data : {
   *(.data) *(.data.*)
   *(.sdata) *(.sdata.*)
   __global_pointer$ = . + 0x800;
} > RAM

.bss : {
   _sbss = .;
   *(.bss) *(.bss.*) *(.sbss) *(.sbss.*) *(COMMON)
   _ebss = .;
} > RAM
```

### 7.4 Krashimi i Vivado gjatë Opt Design (Power Optimization)

Simptoma: implementimi (Synthesis → Implementation) dështonte konsistentisht në fazën opt_design, me gabimin Abnormal program termination

.(EXCEPTION_ACCESS_VIOLATION), pa asnjë mesazh Tcl të dukshëm përpara krashimit.

Diagnoza: analiza e log-ut të plotë (runme.log) zbuloi se krashimi ndodhte specifikisht brenda fazës Power Optimization Task, menjëherë pas një paralajmërimi: WARNING: [Pwropt 34-321] HACOOException: Too many TFIs and TFOs in design. Ky është një limit i njohur i brendshëm i Vivado 2022.2: kur netlist-i ka shumë lidhje me "Transitive Fanin/Fanout" (TFI/TFO) — siç ndodh natyrshëm me dy memorie BRAM 64 KB plus një FSM 5-gjendje me

sinjale stall që ndikojnë pothuajse çdo regjistër të dizajnit — algoritmi i analizës së fuqisë mund të dështojë në mënyrë jo-elegante (segfault native), në vend të një gabimi Tcl të trajtueshëm.

Zgjidhja: u përdor parametri specifik i sugjeruar nga vetë mesazhi i warning-ut, i vendosur përpara çdo launch_runs në Tcl Console të Vivado:

```
set_param pwropt.maxFaninFanoutToNetRatio 100000
```

Ky parametër rrit limitin e brendshëm që pwropt po e kalonte, duke e lejuar analizën e fuqisë të përfundojë normalisht në vend që të dalë nga loop-i dhe të krashojë. Ky episod ilustron një kategori sfide krejt të ndryshme nga bug-et logjike: limit i vetë mjetit të sintezës, jo i dizajnit RTL — diagnostikuar përmes leximit të kujdesshëm të log-ut të plotë, jo thjesht mesazhit të gabimit të shfaqur në GUI.

### 7.5 Loop Infinit në core_bench_state (Bug në Versionin e Thjeshtuar)

Simptoma: në një version të hershëm, të thjeshtuar të core_main.c (përpara kalimit në algoritmin zyrtar EEMBC, shih Kapitullin 7.6), ekzekutimi i CoreMark ndalonte përgjithmonë menjëherë pas mesazhit "[EXEC 3] Nisja e State Machine", pa printuar kurrë mesazhin tjetër.

Diagnoza: thirrja te core_bench_state() i kalonte parametrin step si res->seed1. Me parametrat standardë të validimit CoreMark (seed1=0, seed2=0, seed3=0x66), step=0. Brenda core_state.c, ekziston një loop while (p < memblock+blksize) { ...; p += step; } — me step=0, pointer-i p NUK përparon kurrë, duke prodhuar një loop infinit real në C, jo problem hardware.

Verifikimi: hipoteza u konfirmua empirikisht përmes një testi të dedikuar në C native (jo RISC-V), duke përdorur komandën Linux timeout: thirrja me step=0 u ndal me forcë nga timeout pas 5 sekondash (exit code 124, provë definitive e loop-it infinit), ndërsa thirrja me step=1 përfundoi menjëherë me sukses.

Zgjidhja: zëvendësimi i parametrit step me një konstante jo-zero (fillimisht 1, më pas e zëvendësuar plotësisht nga algoritmi zyrtar EEMBC, shih 7.6, që e shfrytëzon parametrin dtype të derivuar saktë nga vetë të dhënat e listës, me mbrojtje shprehimisht kundër step shumë të vogël).

### 7.6 Nga Algoritmi i Thjeshtuar tek CoreMark Zyrtar EEMBC

Versioni fillestar funksional i core_main.c thërriste core_bench_list(), core_bench_matrix(), dhe core_bench_state() drejtpërdrejt dhe në mënyrë të pavarur nga brenda iterate(), me parametra fiks. Ky version ekzekutohej me sukses dhe demonstronte funksionimin themelor të procesorit, por prodhonte vlera crcmatrix dhe crcstate që NUK përputheshin me tabelat zyrtare EEMBC — sepse algoritmi zyrtar real ndërthur këto tre teste në një mënyrë krejt tjetër: core_bench_list() thërret core_list_mergesort() me një funksion krahasimi (cmp_complex), i cili, bazuar te bitet specifike të çdo nyje liste individuale, vendos VETË nëse dhe si të thërrasë core_bench_matrix() ose core_bench_state(), me parametra të derivuar dinamikisht — jo fiks.

Për të arritur përputhshmëri zyrtarisht të vlefshme (parakusht për çdo krahasim shkencor serioz me platforma të tjera), u klonua direkt repository-ja zyrtare e EEMBC (github.com/eembc/coremark) dhe u verifikuan me checksum MD5 — kundrejt coremark.md5 të publikuar nga vetë EEMBC — pesë skedarët kryesorë algoritmikë:

| Skedari | Statusi i Verifikimit |
|---|---|
| core_list_join.c | MD5 identik me publikimin zyrtar EEMBC |
| core_matrix.c | MD5 identik me publikimin zyrtar EEMBC |
| core_state.c | MD5 identik me publikimin zyrtar EEMBC |
| core_util.c | MD5 identik me publikimin zyrtar EEMBC |
| core_main.c | Logjika algoritmike identike; vetëm mesazhe debug UART shtesë |

*Tabela 9 Verifikimi i integritetit të kodit CoreMark zyrtar përmes checksum MD5.*

Pas këtij zëvendësimi, dhe pas kalibrimit të numrit të iteracioneve për të kapërcyer pragun e 10 sekondave të kërkuar nga vetë CoreMark për një rezultat të vlefshëm, u arrit përputhja e plotë CRC e dokumentuar në Kapitullin 5.4 — provë e fortë e korrektësisë funksionale të plotë të procesorit.

### 7.7 Verifikimi Empirik i Hipotezave: Rasti i Loop-it Infinit

Vlen të theksohet metodologjia specifike e përdorur për të konfirmuar hipotezën e Kapitullit 7.5 (loop infinit në core_bench_state), sepse demonstron një teknikë të përgjithshme e të vlefshme për çdo debug-im hardware-software: kur një hipotezë mund të testohet në mënyrë

të izoluar në C native (pa varësi nga harduer RISC-V), kjo është shumë më e shpejtë dhe më definitive sesa të testohet drejtpërdrejt mbi FPGA ose në simulim RTL të plotë.

```
// Test i izoluar, ekzekutuar NATIVE (x86), jo ne RISC-V:
int main() {
    core_init_state(size, seed1, memblock);

    // Hipoteza: step=0 prodhon loop infinit
    ee_u16 crc0 = core_bench_state(size, memblock,
                                   seed1, seed2, 0, 0);
    printf("step=0: completed crc=0x%04x\n", crc0);
    return 0;
}

$ gcc -o test_state test_state.c core_state.c core_util.c
$ timeout 5 ./test_state
$ echo $?
124   # <- SIGTERM nga 'timeout': konfirmim DEFINITIV i loop-it infinit
```

Kodi i kthimit 124 nga komanda Unix timeout është provë e pakontestueshme: procesi u detyrua të ndalonte me forcë pas 5 sekondash, që do të thotë funksioni core_bench_state() nuk do të kishte përfunduar KURRË vetvetiu. Krahasuar me thirrjen identike por me step=1, që përfundoi menjëherë (më pak se disa milisekonda), kjo eliminon çdo dyshim të mbetur — problemi ishte në logjikën e vetë algoritmit C, jo në ndonjë sjellje delikate apo specifike të hardware-it RISC-V.

Kjo qasje — "izolo problemin nga sa më pak shtresa të sistemit të jetë e mundur" — u përsërit në forma të ndryshme përgjatë gjithë procesit të debug-imit: nga testbench-e minimale Icarus Verilog (Kapitulli 7.2) deri tek teste C native krejt të pavarura nga harduer (ky seksion), çdo herë me synimin për të reduktuar numrin e ndryshoreve të mundshme përpara se të formulohej një konkluzion final.

# Kapitulli 8: Përfundime dhe Punë të Ardhshme

## 8.1 Përmbledhje e Punimit

Ky projekt diplome dokumentoi rrugëtimin e plotë të projektimit, implementimit, dhe validimit të një procesori 32-bit të bazuar në arkitekturën e hapur RISC-V (RV32I, e zgjeruar konceptualisht me M) mbi një platformë FPGA reale. Objektivi kryesor — ndërtimi i një sistemi bare-metal të aftë për të ekzekutuar korniza softuerike komplekse pa ndihmën e një sistemi operativ — u arrit jo thjesht në kuptimin "procesori ekzekuton diçka", por në kuptimin më të rreptë shkencor: rezultati final i benchmark-ut zyrtar EEMBC CoreMark u verifikua të jetë bit-për-bit identik (CRC) me tabelat e publikuara zyrtarisht, një standard validimi që e bën këtë procesor të krahasueshëm drejtpërdrejt me platforma komerciale.

## 8.2 Konkluzionet Teknike

Nga ekzekutimi i suksesshëm i projektit, dhe veçanërisht nga procesi i gjatë i debug-imit (Kapitulli 7), mund të nxjerrim disa konkluzione teknike thelbësore:

- **Modulariteti i RISC-V mundëson debug-im sistematik:** Thjeshtësia relative e setit të instruksioneve RV32I, kombinuar me ndarjen e qartë mes Control Unit (riscv_isa.sv) dhe Datapath (datapath.sv), bëri të mundur izolimin e çdo bug-u (Kapitulli 7) në një modul specifik dhe të kufizuar, pa nevojë për të rishikuar gjithë sistemin për çdo problem.
- **Memoria sinkrone (BRAM) ndryshon thelbësisht arkitekturën single-cycle:** Kalimi nga memorie kombinatoriale (distributed RAM) tek BRAM sinkron — i domosdoshëm për arsye burimesh FPGA dhe timing-u — nuk është thjesht zëvendësim 1-për-1, por kërkon një shtresë shtesë kontrolli (FSM-ja e përshkruar në Kapitullin 3.5), me kosto reale në throughput (CPI > 1), por pa cenuar korrektësinë funksionale nëse implementohet me kujdes.
- **X-propagation është rrezik real, jo thjesht artefakt simulimi:** Problemet e Kapitullit 7.1 dhe 7.2 demonstrojnë se memorie e pa-inicializuar plotësisht (zona jashtë intervalit të firmware-it real) nuk është thjesht "e padëmshme" — mund të prodhojë sjellje të paqëndrueshme që maskohet si probleme krejt të tjera (timer që kthen gjithmonë zero, CPU që ngrin pa shpjegim të dukshëm).
- **Toolchain-i dhe linker-i janë pjesë integrale e "hardware-it":** Shumë nga problemet më të vështira për t'u diagnostikuar (Kapitulli 7.3) nuk ishin në vetë SystemVerilog-un, por në

ndërfaqen mes kompilatorit GCC, linker script-it, dhe sekuencës së zgjimit të procesorit. Për sisteme bare-metal, korrektësia e ABI-t dhe e Global Pointer-it është po aq kritike sa korrektësia e ALU-së.

- **Validimi përmes CoreMark zyrtar (jo një version i thjeshtuar) është thelbësor:** Kalimi nga një implementim i thjeshtuar i benchmark-ut (që ekzekutohej por jepte CRC të gabuara) tek algoritmi zyrtar EEMBC (Kapitulli 7.6) ishte hapi që transformoi rezultatin nga "dëshmi e funksionimit bazë" në "validim shkencërisht i krahasueshëm".

### 8.3 Punët e Ardhshme dhe Rekomandime

Ndonëse procesori aktual është plotësisht funksional dhe i validuar, ekzistojnë hapësira të konsiderueshme për optimizime arkitekturore që mund të shërbejnë si projekte kërkimore vijuese:

- **Aktivizimi i Plotë i Zgjerimit M (Hardware MUL/DIV):** ALU-ja (alu.sv) tashmë implementon plotësisht MUL/DIV/REM/REMU, por firmware-i final u kompilua me -march=rv32i (jo rv32im) për arsye konservatore timing-u (Kapitulli 7.3, 4.1). Një hap i ardhshëm i drejtpërdrejtë do të ishte shtimi i një cikli pritjeje shtesë në FSM (analog me S_LOAD_WAIT) specifikisht për instruksionet DIV/REM, duke lejuar kompilimin me rv32im pa rrezik timing-u, dhe duke rritur ndjeshëm CoreMark Score (sipas analizës së Kapitullit 5.4, ky është faktori kryesor i diferencës me procesorë komercialë).

- **Pipeline i Thjeshtë 2-3 Fazor:** Arkitektura aktuale single-cycle (e zgjeruar me FSM për BRAM) mund të evoluojë drejt një pipeline klasik (Fetch/Decode/Execute, ose Fetch/Execute/Writeback), duke mbivendosur fazat e instruksioneve të njëpasnjëshme dhe duke rritur throughput-in pa rritur frekuencën e orës.

- **Shtimi i Memories Kesh (L1 Cache):** Integrimi i një kesh-i të vogël për instruksione do të reduktonte numrin e cikleve të pritjes BRAM për instruksione të përsëritura (loop-e), duke përmirësuar drejtpërdrejt metrikën CoreMark/MHz.

- **Njësia e Pikës Lëvizëse (FPU):** Zgjerimi i procesorit me setin "F" do të mundësonte llogaritjet me pikë lëvizëse direkt në harduer, duke e bërë sistemin të përshtatshëm për aplikacione më të gjera, si përpunimi i sinjaleve (DSP).

- **Parashikuesi i Degëzimeve (Branch Prediction):** Implementimi i një mekanizmi të thjeshtë parashikimi (p.sh. "gjithmonë jo i marrë" ose 1-bit predictor) do të reduktonte

humbjet e cikleve gjatë degëzimeve të kushtëzuara, veçanërisht të rëndësishme në testin State Machine të CoreMark.

Në përmbledhje, ky punim demonstron se ndërtimi i një procesori RISC-V nga e para — i çuar deri në validim me një benchmark industrial zyrtar — kërkon jo vetëm njohuri solide të arkitekturës së kompjuterave, por gjithashtu rigorozitet metodologjik në diagnostikimin dhe zgjidhjen e problemeve që shfaqen vetëm kur teoria ndeshet me kufizimet praktike të hardware-it real.

### 8.4 Reflektim Final mbi Procesin e Punimit

Përtej rezultatit teknik final, procesi i këtij projekti ofroi mësime metodologjike që shtrihen përtej fushës specifike të dizajnit të procesorëve. E para, dhe ndoshta më e rëndësishmja: asnjë nga problemet e dokumentuara në Kapitullin 7 nuk u zgjidh me "intuitë" ose "provë-gabim" e pastrukturuar — çdo zgjidhje erdhi nga formulimi explicit i një hipoteze ("nëse X është shkaku, atëherë do të vërej Y"), ndjekur nga një test specifik i krijuar pikërisht për të konfirmuar ose hedhur poshtë atë hipotezë. Kjo disiplinë, e huazuar nga metoda shkencore klasike, u tregua më e shpejtë dhe më e besueshme se çdo qasje alternative "eksploruese".

E dyta, vlera e simulimit si hap i ndërmjetëm mes teorisë dhe hardware-it fizik nuk mund të mbivlerësohet. Çdo herë që një problem u përpoq të diagnostikohej direkt mbi FPGA (përmes vëzhgimit të LED-eve ose sjelljes së jashtme), koha e nevojshme për një cikël të plotë "ndrysho-sintetizo-ngarko-testo" (zakonisht 5-15 minuta për shkak të kohës së sintezës Vivado) e bëri procesin dukshëm më të ngadalshëm sesa cikli ekuivalent në simulim (disa sekonda). Kjo përvojë praktike forcoi bindjen se investimi në infrastrukturë testimi (testbench-e të mira, simulim i shpejtë) paguhet shumëfish në kohën totale të zhvillimit.

Së fundi, fakti që disa nga problemet më kohë-konsumuese (Kapitulli 7.3, 7.4) nuk ishin në vetë logjikën digitale të projektuar nga studenti, por në ndërveprimin me mjete të jashtme (kompilatorë, linker-a, vetë Vivado), nënvizon një realitet të rëndësishëm të inxhinierisë moderne të sistemeve të ngulitur: kompetenca në një teknologji të vetme (SystemVerilog, në këtë rast) nuk mjafton — suksesi i projektit kërkoi gjithashtu aftësi në leximin e dokumentacionit teknik të jashtëm, interpretimin e mesazheve gabimi nga mjete komplekse industriale, dhe një qasje sistematike ndaj debug-imit ndër-shtresor (cross-layer debugging) që shtrihet nga porta logjike deri tek skedari Linker.

## Bibliografia

[1] Harris, S. L., Harris, D. *"Digital Design and Computer Architecture: RISC-V Edition"*. Morgan Kaufmann, 2021.

[2] Chu, P. P. *"Embedded RTL Design Using SystemVerilog: An Example-Based Approach for FPGAs"*. Wiley-IEEE Press, 2025.

[3] EEMBC (Embedded Microprocessor Benchmark Consortium). "CoreMark: An EEMBC Benchmark", repository zyrtar: github.com/eembc/coremark, qasur Qershor 2026.

[4] EEMBC. "CoreMark Score Reporting Rules", README.md i repository-së zyrtare CoreMark, EEMBC, 2018.

[5] Xilinx Inc. "Vivado Design Suite User Guide: Synthesis (UG901)", Xilinx/AMD, 2022.

[6] Xilinx Inc. "Vivado Design Suite User Guide: Implementation (UG904)", Xilinx/AMD, 2022.

[7] Digilent Inc. "Nexys A7 FPGA Board Reference Manual", Digilent Reference, 2020.

[8] xPack Project. "xPack RISC-V Embedded GCC", dokumentacion zyrtar: xpack.github.io/dev-tools/riscv-none-elf-gcc, 2024.

[9] Free Software Foundation. "Using the GNU Compiler Collection (GCC): Extended Asm — Assembler Instructions with C Expression Operands", GCC Documentation, 2023.

[10] Free Software Foundation. "GNU Linker (ld) Documentation — Linker Scripts", Binutils Documentation, 2023.

[11] Icarus Verilog Project. "Icarus Verilog", repository: github.com/steveicarus/iverilog, qasur 2026.

[12] IEEE Standards Association. "IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language (IEEE 1800-2017)", IEEE, 2018.

[13] ARM Limited. "Cortex-M Series Processors — CoreMark Performance Data", arm.com/products/silicon-ip-cpu, qasur 2026.

[14] Universiteti Politeknik i Tiranës, Fakulteti i Teknologjisë së Informacionit. "Udhëzues për Hartimin e Punimeve të Diplomës", UPT, 2024.

## Shtojca A: Fragmente nga Kodi SystemVerilog i Komentuar

Gjatë fazës finale të projektit, e gjithë baza e kodit SystemVerilog (13 module) u rikomentua plotësisht, rresht-për-rresht, duke shpjeguar funksionin e çdo sinjali, çdo bllok always_ff/always_comb, dhe arsyetimin arkitekturor pas çdo vendimi dizajni — pa ndryshuar asnjë rresht logjik funksional (verifikuar përmes krahasimi diferencial të kodit pa komente, dhe përmes ri-simulimi të plotë të sistemit pas komentimit). Më poshtë paraqiten fragmente përfaqësuese nga dy module qendrore; kodi i plotë i komentuar gjendet në skedarët shoqërues të këtij punimi.

### A.1 Fragment nga top.sv — FSM-ja e Sinkronizimit

```
// is_load_next: bazuar ne instr_raw (vlera E RE nga BRAM, jo
// instr_latched ende i vjeter!) Perdoret VETEM brenda next-state
// logic te S_FETCH2, kur instr_latched akoma mban instruksionin
// e MEPARSHEM (do te perditesohet ne fund te ketij cikli).
//
// SHENIM (bug i zgjidhur gjate debug-imit): nese ketu do
// perdorej 'is_load' (bazuar te instr_latched, i vjeter),
// next-state logic do te vendoste GABIMISHT bazuar ne
// instruksionin e MEPARSHEM, jo te ai i ardhshem - kjo do te
// shkaktonte X-propagation per instruksionet LOAD (shih Kap. 7.2).
assign is_load_next = (instr_raw[6:0] == 7'b0000011);

always_comb begin
    state_next = state;
    case (state)
        S_FETCH1: state_next = S_FETCH2;
        S_FETCH2: begin
            if (is_load_next) state_next = S_DECODE_LOAD;
            else              state_next = S_EXEC;
        end
        S_DECODE_LOAD: begin
            if (is_ram_load) state_next = S_LOAD_WAIT;
            else             state_next = S_EXEC;
        end
```

```
        S_LOAD_WAIT: state_next = S_EXEC;
        S_EXEC:      state_next = S_FETCH1;
        default:     state_next = S_FETCH1;
    endcase
end
```

**A.2 Fragment nga data_mem.sv — Zero-Inicializimi i BRAM**

```
// RREGULLIM KRITIK: inicializo GJITHE RAM me ZERO se pari.
// memfile.dat permban vetem .text+.rodata+.data (fjalet e
// perdorura nga programi), JO .bss (variablat globale
// jo-inicializuara, si start_time_val/stop_time_val). Pa kete
// hap, fjalet PAS fundit te memfile.dat mbeten 'X' (e
// papercaktuar) ne simulim, dhe ne FPGA real mund te kete
// vlera te paqendrueshme. Kjo X/random e perhapet
// (X-propagation) ne register, ALU, FSM next-state logic, dhe
// CPU ngec/sillet gabim sapo lexon nje variabel globale
// jo-inicializuar (shih Kap. 7.1).
initial begin
    integer i;
    for (i = 0; i < 16384; i = i + 1) begin
        RAM[i] = 32'h00000000;
    end
    $readmemh("memfile.dat", RAM);
end


// Lexim/shkrim SINKRON - nje cikel vonese (BRAM hardware)
always_ff @(posedge clk) begin
    if (addr[31:16] == 16'h0000) begin
        rd <= RAM[addr[15:2]];
        if (we[0]) RAM[addr[15:2]][7:0]   <= wd[7:0];
        if (we[1]) RAM[addr[15:2]][15:8]  <= wd[15:8];
        if (we[2]) RAM[addr[15:2]][23:16] <= wd[23:16];
        if (we[3]) RAM[addr[15:2]][31:24] <= wd[31:24];
    end else begin
```

```
        rd <= 32'h00000000;
    end
end
```

**A.3 Sekuenca e Zgjimit — start.S**

```
.section .text.init
.globl _start
_start:
    /* 1. Inicializimi i Stack Pointer (SP) */
    la sp, _stack_top

    /* 2. Inicializimi i Global Pointer (GP) */
    .option push
    .option norelax
    la gp, __global_pointer$
    .option pop

    /* 3. Inicializimi i Thread Pointer (TP) */
    mv tp, zero

    /* 4. Pastrimi i BSS */
    la t0, _sbss
    la t1, _ebss
    beq t0, t1, .L_bss_done
.L_bss_loop:
    sw   zero, 0(t0)
    addi t0, t0, 4
    bne  t0, t1, .L_bss_loop
.L_bss_done:
    /* 5. Thirr main */
    jal ra, main
.L_loop_forever:
    j .L_loop_forever
```

**A.4 Fragment nga riscv_isa.sv — Dekodimi i Instruksioneve R-type**

```
case(op)
```

```
    // R-type (opcode=0110011): ADD/SUB/MUL/AND/OR/...
    7'b0110011: begin
        reg_write = 1'b1;
        case(funct3)
            // funct3=000: ADD, SUB, ose MUL (M-extension)
            3'b000: alu_control =
                (funct7 == 7'b0000001) ? 5'b01001 :
                (funct7[5]             ? 5'b01000
                                       : 5'b00000);
            3'b001: alu_control = 5'b00001; // SLL
            3'b010: alu_control = 5'b00010; // SLT
            3'b011: alu_control = 5'b00011; // SLTU
            // funct3=100: XOR, ose DIV (M-extension)
            3'b100: alu_control =
                (funct7 == 7'b0000001) ? 5'b10000
                                       : 5'b00100;
            default: alu_control = 5'b00000;
        endcase
    end
endcase
```

**Shtojca B: Lista e Plotë e Skedarëve të Projektit**

Më poshtë listohen të gjithë skedarët burimorë (RTL, software, dhe simulim) që përbëjnë projektin e plotë, të organizuar sipas kategorisë funksionale (shih gjithashtu Kapitullin 6.1 për përshkrimin e strukturës së dosjeve):

- **RTL (SystemVerilog, 13 module):** top.sv, riscv_cpu.sv, riscv_isa.sv, datapath.sv, alu.sv, register_file.sv, imm_extender.sv, pc_reg.sv, instr_mem.sv, data_mem.sv, baud_gen.sv, uart_tx.sv, uart_rx.sv.
- **Software CoreMark (EEMBC zyrtar, MD5-verifikuar):** core_main.c, core_list_join.c, core_matrix.c, core_state.c, core_util.c, coremark.h.

- **Software Porting Layer (i personalizuar për këtë hardware):** core_portme.c, core_portme.h, ee_printf.c, start.S, linker.ld, make_hex.py.
- **Skedarë të Gjeneruar (toolchain output):** firmware.elf, firmware.bin, memfile.dat.
- **Constraints:** nexys_a7.xdc (përcakton pin-at fizikë të Nexys A7-100T: clock 100 MHz, UART TX/RX, LED-të e debug-imit, butoni i reset-it).

## Shtojca C: Shpjegim i Detajuar Modul-për-Modul

Kjo shtojcë plotëson Shtojcën A (që përmban vetëm fragmente të përzgjedhura) me një përshkrim sistematik të TË GJITHA 13 moduleve SystemVerilog të projektit, modul-për-modul. Për secilin modul jepet: (1) qëllimi i tij arkitekturor, (2) tabela e plotë e porteve (input/output) me përshkrimin e secilit sinjal, dhe (3) shpjegimi rresht-pas-rreshti i logjikës kryesore (always_ff/always_comb). Kodi i plotë burimor, me komente brenda vetë file-ve .sv, është dorëzuar bashkë me këtë punim si skedarë shoqërues; këtu fokusohemi në ARSYETIMIN pas çdo vendimi dizajni, jo në thjesht riprodhimin e kodit.

### C.1 top.sv — Moduli Integrues Kryesor

Qëllimi: top.sv është rrënja e gjithë hierarkisë së sintezës (Figura 6.1). Instancion bërthamën e CPU-së (riscv_cpu), të dyja memoriet (instr_mem, data_mem), periferikët UART (uart_tx, uart_rx, baud_gen), dhe implementon Vetë FSM-në 5-gjendje (Kapitulli 3.5, Figura 3.3) që sinkronizon dizajnin single-cycle me memorien BRAM sinkrone.

| Porti | Drejtimi | Përshkrimi |
|---|---|---|
| clk | input | Clock fizik 100 MHz nga FPGA-ja (Nexys A7). |
| cpu_resetn | input | Buton fizik reset, active-LOW. |
| uart_rx_pin / uart_tx_pin | input/output | Pin-at fizikë seriale drejt/nga PC (PuTTY). |
| LED | output [3:0] | 4 LED debug — heartbeat, status UART, status RAM write. |

*Tabela 10: Portet e modulit top.sv.*

Blloqet kryesore: (1) gjenerimi i clk_50 nga toggle flip-flop mbi clk; (2) FSM-ja 5-gjendje (state, state_next), e detajuar plotësisht në Kapitullin 3.5; (3) regjistrat instr_latched/read_data_latched, që 'ngrijnë' vlerat e nevojshme përgjatë fazave të FSM-së; (4) logjika e mbrojtjes UART (tx_busy_or_pending, Kapitulli 3.7), që parandalon dërgimin e dyfishuar të karaktereve; (5) dekoderi MMIO (mmio_read_data), që multiplekson mes UART/Timer/Status për leximet nga adresat 0x80000000+.

### C.2 riscv_cpu.sv — Bërthama e Procesorit

Qëllimi: Lidh Control Unit-in (riscv_isa) me rrugën e të dhënave (datapath), dhe implementon timer-in harduerik (cycle_counter) të përdorur nga CoreMark për matjen e kohës (Kapitulli 5.4).

| Porti | Drejtimi | Përshkrimi |
|---|---|---|
| clk, rst, stall | input | Sinjale kontrolli nga top.sv (stall nga FSM). |
| instr | input [31:0] | Instruksioni aktual (= instr_latched nga top.sv). |
| read_data | input [31:0] | Të dhëna nga memorie/MMIO (për LOAD). |
| pc | output [31:0] | Program Counter aktual (për fetch-in tjetër). |
| alu_result | output [31:0] | Rezultati ALU (= data_addr për LOAD/STORE). |
| write_data | output [31:0] | Të dhëna për STORE. |
| mem_write | output | 1 nëse instruksioni aktual është STORE. |
| timer_val | output [31:0] | Vlera e cycle_counter (timer harduerik). |

*Tabela 11: Portet e modulit riscv_cpu.sv.*

Detaj kritik i zbuluar gjatë debug-imit: instancimi i riscv_isa BRENDA këtij moduli duhet të kalojë EXPLICITISHT 'instr' të plotë (jo vetëm fushat e ndara op/funct3/funct7), sepse decoder-i ka nevojë për bitin instr[30] (i pranishëm vetëm në instruksionin e plotë) për të dalluar SRA/SRL dhe DIVU nga variantet e tjera funct3=101. Pa këtë lidhje, ai bit do të ishte i papërcaktuar (X), duke prodhuar sjellje të gabuar specifikisht për këto instruksione.

### C.3 riscv_isa.sv — Njësia e Kontrollit (Decoder)

Qëllimi: Modul pure kombinatorial (zero regjistra) që lexon opcode/funct3/funct7 dhe gjeneron të gjitha sinjalet e kontrollit të nevojshme nga datapath (Kapitulli 3.4, Tabela 2.2 për kodimin e plotë të alu_control).

Struktura: 1 'case (op)' i jashtëm me 9 degë kryesore (R-type, I-type ALU, LOAD, STORE, BRANCH, JAL, JALR, LUI, AUIPC), secila duke vendosur kombinimin e duhur të reg_write,

alu_src, mem_to_reg, mem_write, branch/jal/jalr, dhe alu_control. Brenda degës R-type dhe I-type ALU, ekziston një 'case (funct3)' i brendshëm që dallon nën-variantet (p.sh. ADD kundër SUB kundër MUL, të gjitha me funct3=000). Të gjitha sinjalet vendosen në vlera DEFAULT (0) në fillim të always_comb, praktikë standarde që parandalon krijimin e latch-eve të paqëllimshme në sintezë.

### C.4 datapath.sv — Rruga e të Dhënave

Qëllimi: Implementon vetë "trupin" e CPU-së single-cycle — PC, shkëmbimin e operandeve me Register File, ALU, dhe multiplekserët e write-back. Është moduli më kompleks për nga numri i sinjaleve të ndërmjetme (Kapitulli 3.2, Figura 3.2).

| Blloku | Funksioni |
|---|---|
| always_ff (PC) | Regjistron pc_next në pc, vetëm kur !stall. Reset asinkron kthen pc=0. |
| always_comb (branch_taken) | Vendos nëse kushti BEQ/BNE/BLT/BGE/BLTU/BGEU është i vërtetë, bazuar te zero/cmp_result nga ALU. |
| always_comb (aligned_wd) | Përgatit të dhënat për STORE — replikon byte/half-word për SB/SH. |
| always_comb (loaded_data) | Ekstrakton dhe shenjon/zero-shtrin byte/half-word për LB/LH/LBU/LHU nga fjala e plotë e lexuar. |
| always_comb (result) | Multiplekseri final i write-back — zgjedh mes loaded_data, pc_plus4, imm_ext, ose alu_result. |

*Tabela 12 Blloqet kryesore logjike të datapath.sv.*

### C.5 alu.sv — Njësia Aritmetike-Logjike

Qëllimi: Pure kombinatorial, kryen të gjitha operacionet aritmetike/logjike dhe (M-extension) MUL/DIV/REM/REMU, sipas kodimit alu_control (Tabela 2.2). Çdo operacion division mbrohet shprehimisht kundër pjesëtimit me zero, sipas specifikimit RISC-V (jo trap/exception, por vlera të percaktuara: DIV/DIVU→gjithë-1, REM/REMU→operandi origjinal).

### C.6 register_file.sv — Banka e Regjistrave

Qëllimi: 32 regjistra 32-bit (x0-x31), me LEXIM asinkron (2 porte, rs1/rs2) dhe SHKRIM sinkron (1 portë, rd). Regjistri x0 është i 'hardkoduar' në 0 në dy nivele mbrojtjeje: (1) leximi e detyron explicit (assign rd1 = (rs1==0) ? 0 : registers[rs1]), dhe (2) shkrimi e ndalon explicit (if (reg_write && rd!=0)). Ky dualitet mbrojtjeje siguron korrektësinë e x0 pavarësisht se çfarë do të ndodhte teorikisht me array-in e brendshëm.

### C.7 imm_extender.sv — Zgjeruesi i Vlerave Immediate

Qëllimi: Rikonstrukton vlerat immediate nga pozicionet e ndryshme të biteve sipas 5 formateve (I/S/B/J/U, Figura 2.1), dhe i shenjon-shtrin në 32-bit duke replikuar gjithmonë instr[31] (bitin më të lartë të instruksionit origjinal, që në RISC-V është GJITHMONË biti i shenjës së immediate-it, pavarësisht formatit).

### C.8 pc_reg.sv — Modul i Pa-përdorur

Shenim i rëndësishëm dokumentimi: ky modul EKZISTON në bazën e kodit, por NUK instanciohet asgjëkundi (jo në datapath.sv, jo në riscv_cpu.sv, jo në top.sv). Logjika e PC është implementuar drejtpërdrejt brenda datapath.sv (Kapitulli C.4). pc_reg.sv mbetet si artefakt nga një fazë e hershme e dizajnit kur PC mund të ishte planifikuar si modul i veçantë, dhe nuk ndikon në sjelljen e sintetizuar (Vivado nuk e përfshin në netlist nëse nuk instanciohet).

### C.9 instr_mem.sv — Memoria e Instruksioneve

Qëllimi: BRAM 64 KB, lexim SINKRON (1 cikël vonesë), e zero-inicializuar plotësisht para $readmemh (Kapitulli 7.1 — fix kritik për X-propagation). Atributi (* ram_style = "block" *) e detyron Vivado-n të përdorë burime BRAM të dedikuara, jo LUT-based distributed RAM.

### C.10 data_mem.sv — Memoria e të Dhënave

Qëllimi: Analog me instr_mem.sv, por mbështet LEXIM DHE SHKRIM, me granularitet byte (4 bit 'we', një për çdo byte të fjalës 32-bit) për të mbështetur SB/SH/SW. Mbrojtja addr[31:16]==0 izolon RAM-in nga hapësira MMIO (0x80000000+), që trajtohet ndaras brenda top.sv.

### C.11 baud_gen.sv — Gjeneruesi i Baud Rate

Qëllimi: Numerues i thjeshtë që prodhon impulsin 's_tick' me frekuencën Frekuenca_Clock/(N), ku N=326 (Kapitulli 4.6.1) prodhon ~9585.9 baud (0.15% devijim nga 9600 standarde, brenda tolerancës normale UART).

### C.12 uart_tx.sv — Transmetuesi Serial

Qëllimi: FSM 4-gjendje (idle/start/data/stop) që transmeton 1 byte në format standarde 8N1 (1 start bit + 8 data bit LSB-first + 1 stop bit). Sinjali tx_busy = (state != idle) është kritik për mekanizmin e mbrojtjes nga race condition në top.sv (Kapitulli 3.7).

### C.13 uart_rx.sv — Marrësi Serial

Qëllimi: FSM analoge me uart_tx.sv, por për marrjen e të dhënave. Shenim i rëndësishëm: në këtë projekt, 'rx_done_tick' NUK lidhet në top.sv (instancimi është '.rx_done_tick()' — portë e palidhur) — pra funksionaliteti RX ekziston në hardware, por softueri (CoreMark) nuk e

shfrytëzon aktivisht, sepse komunikimi në këtë projekt është kryesisht në një drejtim (CPU → PuTTY).